

EXAMPLE 4.1 THE NEURON.PY PROGRAM

```
#Example 4.1 Neuron.py

from numpy import exp, array, random, dot

#Set up parameters =====
inputs = array([[0, 0], [1, 1], [1, 0], [0, 1]])
outputs = array([[0, 1, 1, 1]]).T
random.seed(1)
weights = 2 * random.random((2, 1)) - 1

#Define a Single Neuron function =====
def neuron(inputs, weights):
    output = 1 / (1 + exp(-(dot(inputs, weights))))
    return output

#Train the Neuron =====
for iteration in range(50000):
    output = 1 / (1 + exp(-(dot(inputs, weights))))
    weights += dot(inputs.T, (outputs - output) * output *
(1 - output))

#Test the Neuron =====
x = array([1, 0])
print (neuron (x,weights))
```

EXERCISE 4.1

Modify Python program from the Example 4.1 so that it performs a logical OR of the inputs.

Example 4.2 shows a Python multiple layer perceptron using the Scikit-Learn library. It has two inputs and one output and performs the logical AND of the inputs.

EXAMPLE 4.2 THE MLP.PY PROGRAM

```
#Example 4.2 Multiple Layer Perceptron
#https://scikit-learn.org/stable/modules/neural_networks_supervised
.html
from sklearn.neural_network import MLPClassifier
X = [[0., 0.], [1., 1.], [0., 1.], [1., 0.]]
y = [0, 1, 1, 1]
```

```

clf = MLPClassifier(solver='lbfgs', alpha=1e-5,
                   hidden_layer_sizes=(5, 2), random_state=1)
clf.fit(X, y)
print(clf.predict([[2., 2.], [-1., -2.])))
print([coef.shape for coef in clf.coefs_])

```

EXERCISE 4.2

Modify the Python program from Example 4.2 so that it has three inputs, instead of two inputs.

Example 4.3 shows a simple Keras multiple layer perceptron for the breast cancer classification. It has an input layer, a hidden layer (dense layer) of 10 neurons, and an output layer. The dense layer is a neural network layer that each neuron in the dense layer receives input from all neurons of its previous layer.

EXAMPLE 4.3 THE `MPL2.PY` PROGRAM

```

#Example 4.4 Multiple Layer Perceptron Classification
from keras.models import Sequential
from keras.layers import Activation, Dense
from keras import optimizers
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

whole_data = load_breast_cancer()
X_data = whole_data.data
y_data = whole_data.target
X_train, X_test, y_train, y_test = train_test_split(X_data, y_data,
                                                    test_size = 0.3, random_state = 7)

features = X_train.shape[1]

#MPL 3-layer Model =====
model = Sequential()
model.add(Dense(10, activation='relu', input_shape=[X_train
                                                    .shape[1]]))
model.add(Dense(10, activation='relu'))
model.add(Dense(1))
print(model.summary())
print(model.get_config())

model.compile(optimizer = 'adam', loss = 'mean_squared_error',
              metrics = ['mse'])

```

```
model.fit(X_train, y_train, batch_size = 50, validation_split=0.2,
epochs = 100, verbose = 1)

results = model.evaluate(X_test, y_test)

print('loss: ', results[0])
print('accuracy: ', results[1])
```

EXERCISE 4.3

Modify the Python program from Example 4.3 so that it uses five neurons for the dense layer, instead of 10 neurons. Compare the performances, such as the training time and the accuracy.

For more information about traditional neural network, also called multiple layer perceptron, check the following links:

```
https://scikit-learn.org/stable/modules/neural\_networks\_  
supervised.html  
https://scikit-neuralnetwork.readthedocs.io/en/latest/module\_  
mlp.html  
https://docs.oracle.com/javase/8/docs/technotes/guides/swing/index  
.html  
https://keras.io/guides/sequential\_model/
```

4.3 Convolutional Neural Networks

The convolutional neural network (CNN, or ConvNet) is probably the most widely used deep learning neural network. CNN is mainly used for image analysis, e.g., image classification, object detection, and image segmentation. It can also be used in recommendation systems, natural language processing, brain-computer interfaces, and financial time series. To date, a number of CNNs have been developed, such as LeNet, AlexNet, GoogLeNet (now Inception), VGG, ResNet, DenseNet, MobileNet, EffecientNet, YOLO, and so on.

Figure 4.6 shows a typical architecture of convolutional neural network that contains an input layer, convolutional layers, pooling layers (subsampling, or down sampling), activation layers, fully connected layers, and an output layer.

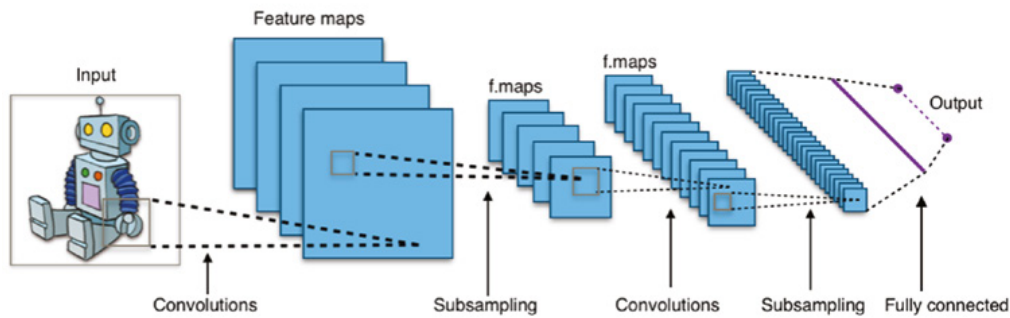


Figure 4.6: Typical convolutional neural network architecture

(Source: https://en.wikipedia.org/wiki/Convolutional_neural_network)

Input Layer This is the layer that takes images as input. It typically has a size of $32 \times 32 \times 1$ for handwriting digit images in grayscale, and $224 \times 224 \times 3$ for color photo images.

Convolutional Layer This is the core structure of a convolutional neural network. It uses feature filters (also called *kernels*) to extract feature information from the input. Figure 4.7a shows some commonly used image filters, or image kernels, that can perform sharpening, blurring, and edge detection of an input image. Figure 4.7b shows the convolution process of an image and a kernel. The convolution process is essentially to multiply the image matrix values with kernel values as an element-wise product and then add all the values to get the convoluted results.

Pooling Layer Pooling layer is another core structure of CNN. The pooling layer is used for downsampling. Max pooling is the most commonly used pooling. As shown in Figure 4.8, max pooling divides the input image into a series of nonoverlapping rectangles and outputs the maximum for each of these subregions. In this example, a $4 \times 48 \times 8$ image has been mapped by a 2×2 window with the stride size of 2. Within the 2×2 window, each region (2×2) is collapsed into a single pixel by choosing the maximum value. In this way, the entire 4×4 image is downsampled into a 2×2 image. Another commonly used pooling is average pooling.

Activation Layer The activation function of a node defines the output of that node as a function of an input or a set of inputs. There are three commonly used activation functions: Rectified Linear Unit (ReLU), hyperbolic tangent, and sigmoid function, as shown in Figure 4.9. In deep learning, ReLU is often preferred because it makes neural network training much faster.

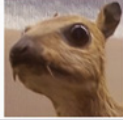
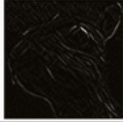
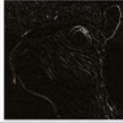
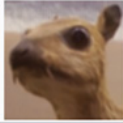
Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	

Figure 4.7a: Commonly used image convolution filters (kernels)

(Source: [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing)))

Fully Connected Layer After several convolutional layers and max pooling layers, the high-level reasoning in the neural network is done via fully connected layers. The neurons in a fully connected layer have connections to all activations in the previous layer, as shown in regular (nonconvolutional) artificial neural networks. Fully connected layer is also called *dense layer*.

Dropout Layer The dropout layer drops out some of nodes in the neural network to prevent overfitting. Dropout layer can be used with most types of layers, typically after the fully connected layer.

Output Layer This layer gives the final output of the CNN. For classification, neural networks the number of outputs depends on the number of classes. For regression neural networks, there is only one output that is a floating-point number, a number with a decimal point.

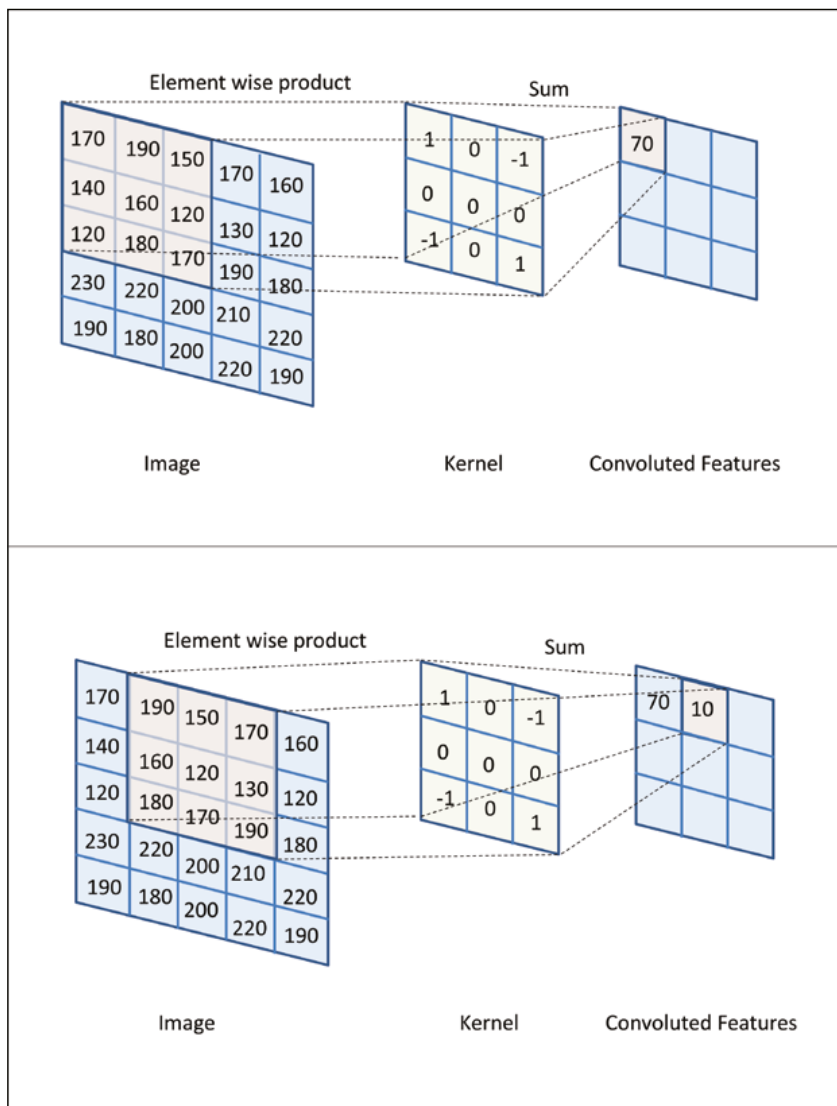


Figure 4.7b: The convolution process of an image and a kernel

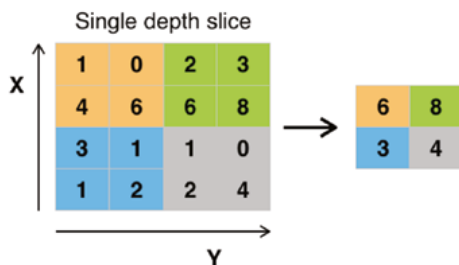


Figure 4.8: Max pooling with a 2x2 filter and stride = 2

(Source: https://en.wikipedia.org/wiki/Convolutional_neural_network)

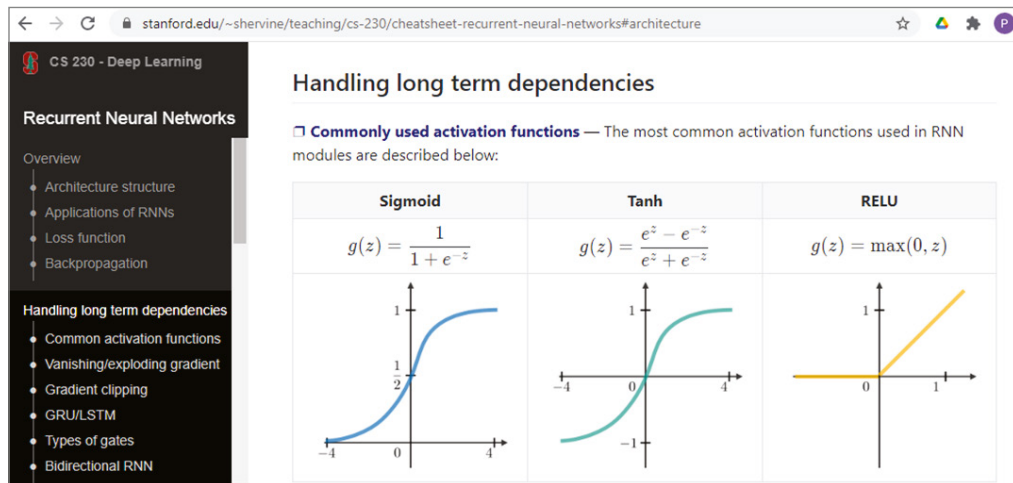


Figure 4.9: Commonly used activation functions

(Source: <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks#architecture>)

The following link shows a simple CNN example implemented with Keras.

<https://github.com/buomsoo-kim/Easy-deep-learning-with-Keras/blob/master/2.%20CNN/1-Basic-CNN/1-basic-cnn.py>

4.3.1 LeNet, AlexNet, GoogLeNet

LeNet, often referred to as LeNet-5, is probably the best known first convolutional neural network. Developed by Yann LeCun and his colleagues in 1989, LeNet is a seven-layer convolutional neural network. LeNet is famous for classifying handwritten digits, and input images must have the size of 28 (height) $\times$ 28 (width) $\times$ 1 (channel). See the following LeNet official website details.

<http://yann.lecun.com/exdb/lenet/>

AlexNet is a convolutional neural network, developed by Alex Krizhevsky in collaboration with Ilya Sutskever and Geoffrey Hinton. Krizhevsky was then a PhD research student of Geoffrey Hinton.

AlexNet made a name for itself when it won the ImageNet Large Scale Visual Recognition Challenge on September 30, 2012. It achieved a top-five error rate of 15.3 percent, more than 10 percentage points lower than the runner-up.

AlexNet has eight layers; the first five were convolutional layers, followed by max-pooling layers, and the last three were fully connected layers. It is divided into two branches, designed for two GPUs. It also uses the nonsaturating ReLU activation function, which showed better training performance than the hyperbolic tangent (Tanh) and sigmoid activation functions. AlexNet has a total of about 61 million parameters and more than 600 million connections.

Figure 4.10 shows the AlexNet architecture from the missinglink.ai website. AlexNet takes input images with the size of 224 (height) $\times$ 224 (width) $\times$ 3 (channel).

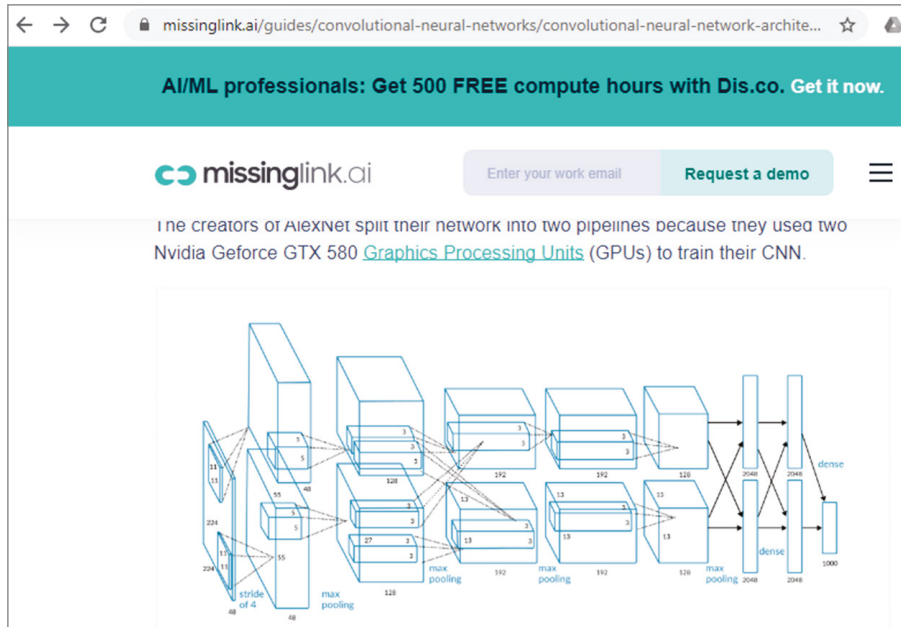


Figure 4.10: The AlexNet architecture from missinglink.ai website

(Source: <https://missinglink.ai/guides/convolutional-neural-networks/convolutional-neural-network-architecture-forging-pathways-future/>)

See the following link for the AlexNet original paper, which is considered one of the most influential papers in computer vision. The AlexNet original paper has been cited more than 70,000 times according to Google Scholar.

<https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf>

GoogLeNet, now known as Inception, is another impressive deep learning neural network. Inception v3 is the third edition of Google's Inception convolutional neural network. GoogLeNet takes input images with the size of 224 (height) $\times$ 224 (width) $\times$ 3 (channel), while Inception takes input images with the size of 299 (height) $\times$ 299 (width) $\times$ 3 (channel).

Compared to AlexNet, Inception has a concept of Inception modules that perform different-sized convolutions and concatenate the filters for the next layer, as shown in Figure 4.11. Inception v3 has 48 layers and 24 million parameters. Although Inception has more layers than AlexNet, it has many fewer parameters.

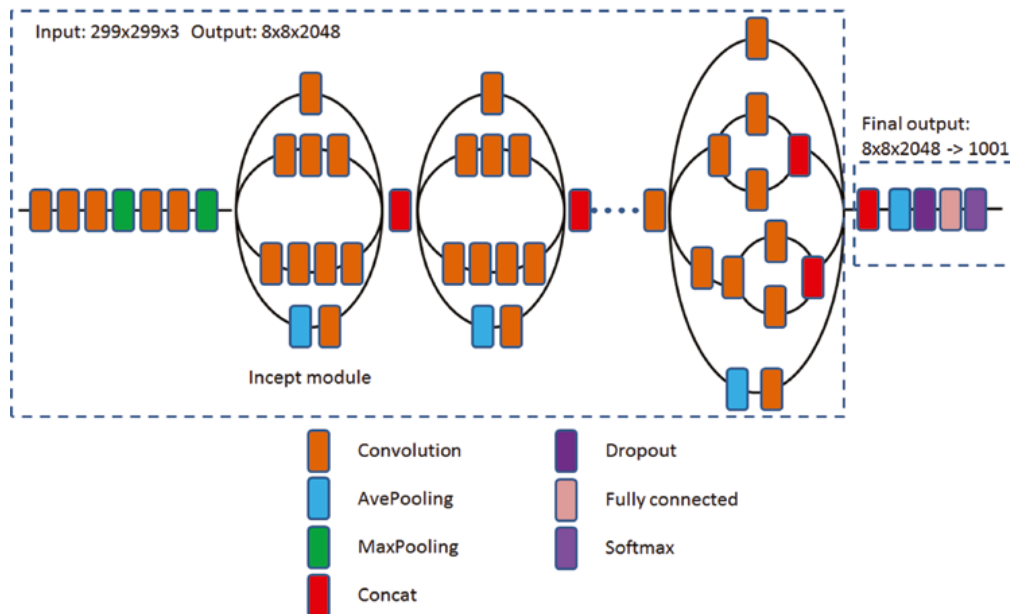


Figure 4.11: The schematic diagram of the Inception v3 architecture

For more details, check the GoogLeNet original paper:

<https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/43022.pdf>
<https://cloud.google.com/tpu/docs/inception-v3-advanced>

Example 4.4 shows a Keras implementation of the LeNet-5 deep learning neural network and the output of the summary of the model.

EXAMPLE 4.4 THE LENET.PY PROGRAM

```
#Example 4.4 LeNet-5 Keras
from keras.models import Sequential
from keras.layers import Dense, Conv2D, Flatten, AveragePooling2D
from keras import optimizers

model = Sequential()
model.add(Conv2D(filters=6, kernel_size=(3, 3), activation='relu',
input_shape=(32,32,1)))
model.add(AveragePooling2D())
model.add(Conv2D(filters=16, kernel_size=(3, 3), activation='relu'))
model.add(AveragePooling2D())
model.add(Flatten())
model.add(Dense(units=120, activation='relu'))
model.add(Dense(units=84, activation='relu'))
model.add(Dense(units=10, activation = 'softmax'))
model.summary()
```

The following is the output:

Using TensorFlow backend.

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 30, 30, 6)	60
average_pooling2d_1 (Average)	(None, 15, 15, 6)	0
conv2d_2 (Conv2D)	(None, 13, 13, 16)	880
average_pooling2d_2 (Average)	(None, 6, 6, 16)	0
flatten_1 (Flatten)	(None, 576)	0
dense_1 (Dense)	(None, 120)	69240
dense_2 (Dense)	(None, 84)	10164
dense_3 (Dense)	(None, 10)	850
Total params: 81,194		
Trainable params: 81,194		
Non-trainable params: 0		

EXERCISE 4.4

Modify Python LeNet program from the Example 4.4 to use a different number of filters and see how this affects the total number of parameters.

Example 4.5 shows a Keras implementation of the AlexNet deep learning neural network and the output of the summary of the model.

EXAMPLE 4.5 THE ALEXNET.PY PROGRAM

```
#Example 4.5 AlexNet with Keras
#Modified from:
#https://engmrk.com/alexnet-implementation-using-keras/

import keras
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten, Conv2D,
MaxPooling2D
from keras.layers.normalization import BatchNormalization
```

```
#Create the AlexNet model
model = Sequential()

# 1st Convolutional Layer
model.add(Conv2D(filters=96, input_shape=(224,224,3), kernel_
size=(11,11), activation='relu', strides=(4,4), padding='valid'))
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2),
padding='valid'))

# 2nd Convolutional Layer
model.add(Conv2D(filters=256, kernel_size=(11,11), activation='relu',
strides=(1,1), padding='valid'))
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2),
padding='valid'))

# 3rd Convolutional Layer
model.add(Conv2D(filters=384, kernel_size=(3,3), activation='relu',
strides=(1,1), padding='valid'))

# 4th Convolutional Layer
model.add(Conv2D(filters=384, kernel_size=(3,3), activation='relu',
strides=(1,1), padding='valid'))

# 5th Convolutional Layer
model.add(Conv2D(filters=256, kernel_size=(3,3), activation='relu',
strides=(1,1), padding='valid'))
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2),
padding='valid'))

# 1st Fully Connected layer
model.add(Flatten())
model.add(Dense(4096, activation='relu', input_shape=(224*224*3,)))
model.add(Dropout(0.4))

# 2nd Fully Connected Layer
model.add(Dense(4096,activation='relu'))
model.add(Dropout(0.4))

# 3rd Fully Connected Layer
model.add(Dense(1000,activation='relu'))
model.add(Dropout(0.4))

# Output Layer
model.add(Dense(17,activation='softmax'))

model.summary()

# Compile the model
model.compile(loss=keras.losses.categorical_crossentropy,
optimizer='adam', metrics=["accuracy"])
```

```
# Fit the model
#model.fit()

# Prediction with the model
#model.evaluate()
```

The following is the output:

Using TensorFlow backend.

Model: "sequential_1"

Layer (type)	Output Shape	Param #
=====		
conv2d_1 (Conv2D)	(None, 54, 54, 96)	34944
max_pooling2d_1 (MaxPooling2D)	(None, 27, 27, 96)	0
conv2d_2 (Conv2D)	(None, 17, 17, 256)	2973952
max_pooling2d_2 (MaxPooling2D)	(None, 8, 8, 256)	0
conv2d_3 (Conv2D)	(None, 6, 6, 384)	885120
conv2d_4 (Conv2D)	(None, 4, 4, 384)	1327488
conv2d_5 (Conv2D)	(None, 2, 2, 256)	884992
max_pooling2d_3 (MaxPooling2D)	(None, 1, 1, 256)	0
flatten_1 (Flatten)	(None, 256)	0
dense_1 (Dense)	(None, 4096)	1052672
dropout_1 (Dropout)	(None, 4096)	0
dense_2 (Dense)	(None, 4096)	16781312
dropout_2 (Dropout)	(None, 4096)	0
dense_3 (Dense)	(None, 1000)	4097000
dropout_3 (Dropout)	(None, 1000)	0
dense_4 (Dense)	(None, 17)	17017
=====		
Total params: 28,054,497		
Trainable params: 28,054,497		
Non-trainable params: 0		

EXERCISE 4.5

Modify the Python AlexNet program from Example 4.5, to use a different number of dense layer units and see how this affects the total number of parameters.

Example 4.6 shows a Python code that can load Google Inception V3 and show the summary of the models using the Keras built-in functions. It is just three lines of code!

EXAMPLE 4.6 THE INCEPT.PY PROGRAM

```
#Example 4.6
from tensorflow.keras.applications import inception_v3

# init the models
model = inception_v3.InceptionV3(weights='imagenet')
print(model.summary())
```

The following is the truncated output, which shows the first few layers and the last few layers:

```
Model: "inception_v3"

```

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, 299, 299, 3)]	0	
conv2d (Conv2D)	(None, 149, 149, 32)	864	input_1[0][0]
...	...	...	...
mixed10 (Concatenate)	(None, 8, 8, 2048)	0	activation_85[0][0] mixed9_1[0][0] concatenate_1[0][0] activation_93[0][0]
avg_pool (GlobalAveragePooling2)	(None, 2048)	0	mixed10[0][0]
predictions (Dense)	(None, 1000)	2049000	avg_pool[0][0]

```

Total params: 23,851,784
Trainable params: 23,817,352
Non-trainable params: 34,432
None

```

Example 4.7 shows a Python code that you can use to create a simple custom-built deep learning neural network and output the summary of the model. It contains an input layer (28, 28, 1), a convolution layer, a max-pooling layer, a dropout layer, a flatten layer, a dense layer, and an output layer. The dense layer is a neural network layer that each neuron in the dense layer receives input from all neurons of its previous layer. The flatten layer flattens the data into a one-dimensional array, which is typically used before the dense layer in convolutional neural networks.

EXAMPLE 4.7 THE `DLMODEL.PY` PROGRAM

```
#Example 4.7

from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten
from keras.layers.convolutional import Conv2D, MaxPooling2D
# Create model
model = Sequential()
model.add(Conv2D(32, (5, 5), input_shape=(28, 28, 1),
activation='relu'))
model.add(MaxPooling2D())
model.add(Dropout(0.2))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(2, activation='softmax'))
# Compile model
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
print(model.summary())
```

This is the output of the example, showing the summary of the customized neural network:

```
Using TensorFlow backend.
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
=====		
conv2d_1 (Conv2D)	(None, 24, 24, 32)	832
max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 32)	0
dropout_1 (Dropout)	(None, 12, 12, 32)	0
flatten_1 (Flatten)	(None, 4608)	0
dense_1 (Dense)	(None, 128)	589952

```
dense_2 (Dense)                (None, 2)                258
=====
Total params: 591,042
Trainable params: 591,042
Non-trainable params: 0
_____
None
```

EXERCISE 4.6

Modify the Python program from Example 4.7, and add another Conv2D, MaxPooling, and Dropout layer before the Flatten layer. Compare the number of parameters.

Example 4.8 shows Python code for loading the MNIST handwriting images and displaying them.

EXAMPLE 4.8 THE MNIST.PY PROGRAM

```
#Example 4.8

from keras.datasets import mnist
import matplotlib.pyplot as plt

# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# plot 4 images as gray scale
plt.subplot(141)
plt.imshow(X_train[0])
plt.subplot(142)
plt.imshow(X_train[1])
plt.subplot(143)
plt.imshow(X_train[2])
plt.subplot(144)
plt.imshow(X_train[3])
plt.show()
```

Figure 4.12 shows a few sample images of MNIST handwriting images.

Example 4.9 shows an improved version of the previous code. It creates a simple deep learning neural network, which is used to classify MNIST handwriting digits.

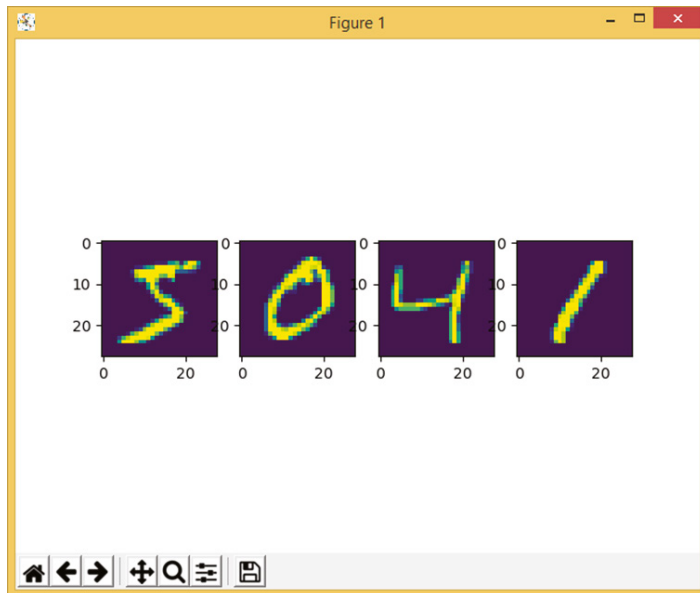


Figure 4.12: The sample images of MNIST handwriting digits images

EXAMPLE 4.9 THE MNIST2.PY PROGRAM

```
#Example 4.9

# Simple CNN for the MNIST Dataset
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten
from keras.layers.convolutional import Conv2D, MaxPooling2D
from keras.utils import np_utils

# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# plot 4 images as gray scale
plt.subplot(141)
plt.imshow(X_train[0])
plt.subplot(142)
plt.imshow(X_train[1])
plt.subplot(143)
plt.imshow(X_train[2])
plt.subplot(144)
plt.imshow(X_train[3])
plt.show()

# reshape to be [samples] [width] [height] [channels]
X_train = X_train.reshape((X_train.shape[0], 28, 28, 1))
X_train = X_train.astype('float32')
X_test = X_test.reshape((X_test.shape[0], 28, 28, 1))
X_test = X_test.astype('float32')
```

```

# normalize inputs from 0-255 to 0-1
X_train = X_train / 255
X_test = X_test / 255
# one hot encode outputs
y_train = np_utils.to_categorical(y_train)
y_test = np_utils.to_categorical(y_test)
num_classes = y_test.shape[1]
# Create model
model = Sequential()
model.add(Conv2D(32, (5, 5), input_shape=(28, 28, 1),
activation='relu'))
model.add(MaxPooling2D())
model.add(Dropout(0.2))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))
# Compile model
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])

# Fit the model
model.fit(X_train, y_train, validation_data=(X_test, y_test),
epochs=10, batch_size=200)
# Evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("CNN Error: %.2f%%" % (100-scores[1]*100))

```

This is the output of the example, showing the summary of the neural network:

Using TensorFlow backend.
Model: "sequential_1"

Layer (type)	Output Shape	Param #
=====		
conv2d_1 (Conv2D)	(None, 24, 24, 32)	832

max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 32)	0

dropout_1 (Dropout)	(None, 12, 12, 32)	0

flatten_1 (Flatten)	(None, 4608)	0

dense_1 (Dense)	(None, 128)	589952

dense_2 (Dense)	(None, 2)	258
=====		
Total params: 591,042		
Trainable params: 591,042		
Non-trainable params: 0		

None		

EXERCISE 4.7

Modify the Python program from Example 4.9, change the kernel size of 32 to 64 in the Conv2D layer, and compare their performances. Change the filter size (5,5) to (3,3) and compare the performances.

EXERCISE 4.8

Modify the Example 4.9 Python program, add another Conv2D, MaxPooling, and Dropout layer before the flatten layer. Compare the performances.

4.3.2 VGG, ResNet, DenseNet, MobileNet, EffecientNet, and YOLO

The VGG deep learning neural network was developed by the Visual Geometry Group at the University of Oxford. There are two versions, VGG 16 and VGG 19. VGG 16 is a 16-layer architecture with a pair of convolutional layers, a pooling layer, and fully connected layer at the end. A VGG network is the idea of much deeper networks and with much smaller filters. A VGG takes input images with the size of 224 (height) $\times$ 224 (width) $\times$ 3 (channel).

The following link shows an interesting VGG practice website from the University of Oxford, where you can learn to build VGG networks step-by-step using MATLAB software (www.mathworks.com).

<https://www.robots.ox.ac.uk/~vgg/practicals/cnn/index.html>

Figure 4.13 shows the architecture of VGG16 neural networks. It has an input of $224 \times 224 \times 3$, followed by a convolutional layer and a ReLU layer that reduce the size to $224 \times 224 \times 64$, then to $112 \times 112 \times 128$, $56 \times 56 \times 256$, and finally to $1 \times 1 \times 1000$.

A **ResNet** (Residual Network) is a convolutional neural network developed by Microsoft. It took the deep learning world by storm in 2015, as the first neural network that could train hundreds or thousands of layers without succumbing to the “vanishing gradient” problem.

ResNet builds on constructs known from pyramidal cells in the cerebral cortex. This is done by utilizing jump connections or shortcuts to skip some layers. Typical ResNet models are implemented with double- or triple-layer jumps that contain nonlinearities (ReLU) and batch normalization in between; see Figure 4.14. ResNet takes input images with the size of 224 (height) $\times$ 224 (width) $\times$ 3 (channel).

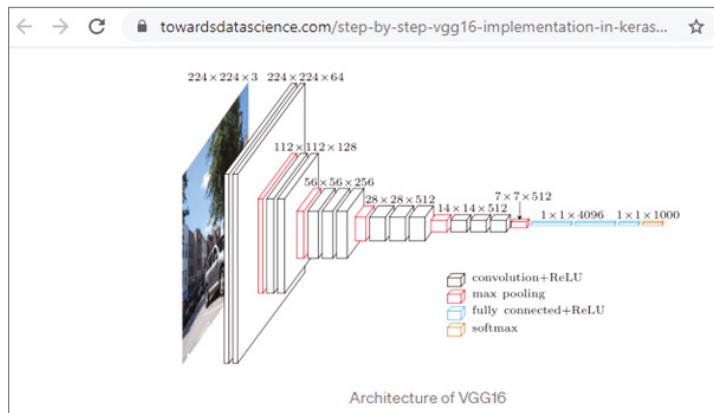


Figure 4.13: The architecture of VGG16 neural networks

(Source: <https://towardsdatascience.com/step-by-step-vgg16-implementation-in-keras-for-beginners-a833c686ae6c>)

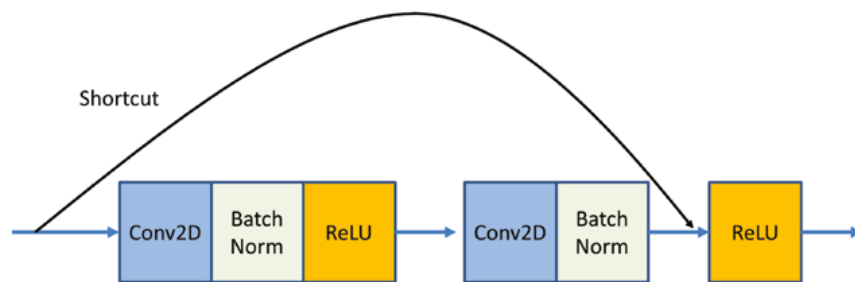


Figure 4.14: The schematic diagram of the ResNet structure with a double layer skip

For more details of ResNet, see the following:

<https://arxiv.org/abs/1512.03385v1>

<https://paperswithcode.com/method/resnet>

DenseNet is a convolutional neural network that uses dense connections between layers, where all layers are directly connected; see Figure 4.15. DenseNet has several compelling advantages over other neural networks. It can mitigate the vanishing-gradient problem, enhance feature propagation, promote feature reuse, and significantly reduce the number of parameters. DenseNet takes input images with the size of 224 (height) $\times$ 224 (width) $\times$ 3 (channel).

For more details of DenseNet, see the following:

<https://arxiv.org/abs/1608.06993>

<https://paperswithcode.com/method/densenet>

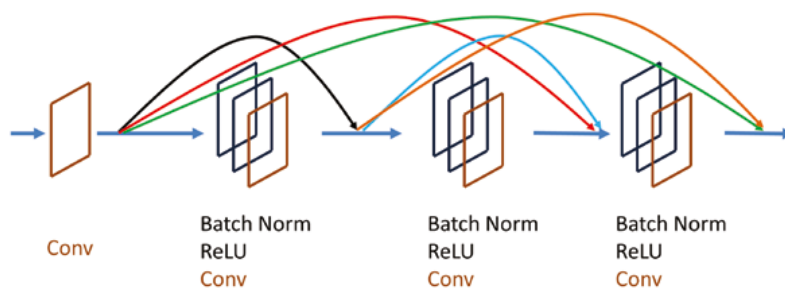


Figure 4.15: The schematic diagram of the DenseNet structure with connections between layers

MobileNet is a family of general-purpose convolutional neural networks designed for use on mobile devices. MobileNet comes in two versions and can be used for image classification, object detection, and more. MobileNets are based on a streamlined architecture for building lightweight deep neural networks. Two simple global hyper-parameters are introduced to find an efficient trade-off between latency and accuracy. These hyper-parameters allow modelers to choose the right model for their applications.

Figure 4.16 shows an overview of MobileNetV1 and the structure of Depthwise Separable Convolution (Light Weight Model). MobileNet takes input images with the size of 224 (height) $\times$ 224 (width) $\times$ 3 (channel).

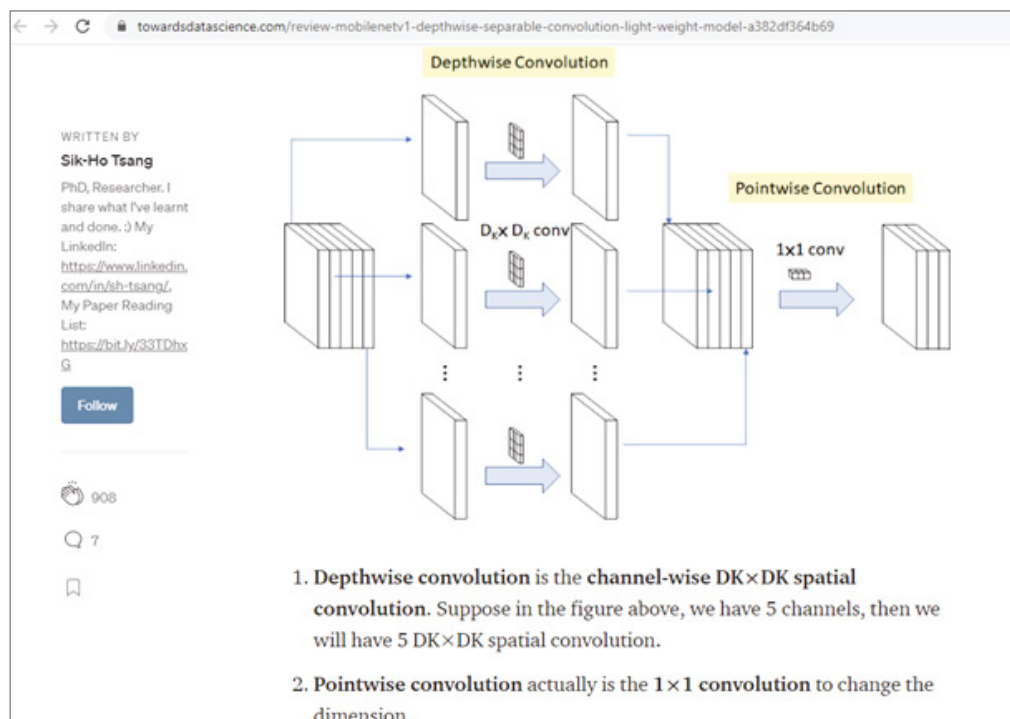


Figure 4.16: Review: MobileNetV1 — Depthwise Separable Convolution (Light Weight Model)

(Source: <https://towardsdatascience.com/review-mobilenetv1-depthwise-separable-convolution-light-weight-model-a382df364b69>)

For more details of MobileNet, see the following:

<https://arxiv.org/abs/1704.04861>

<https://ai.googleblog.com/2019/11/introducing-next-generation-on-device.html>

<https://github.com/tensorflow/models/tree/master/research/slim/nets/mobilenet>

EfficientNet is a relatively new deep learning neural network developed by Mingxing Tan, at Google AI, in May 2019. It was developed based on AutoML and Compound Scaling. First, a mobile-size baseline network called EfficientNet-B0 was developed using AutoML MNAS Mobile framework. Then, the compound scaling method was used to scale up this baseline to obtain EfficientNet-B1 to B7. EfficientNet has achieved much better accuracy and efficiency than previous convolutional neural networks. EfficientNet-B7 has achieved a top-one accuracy of 84.3 percent on ImageNet; see Figure 4.17. Efficient takes input images with the size of 224 (height) $\times$ 224 (width) $\times$ 3 (channel),

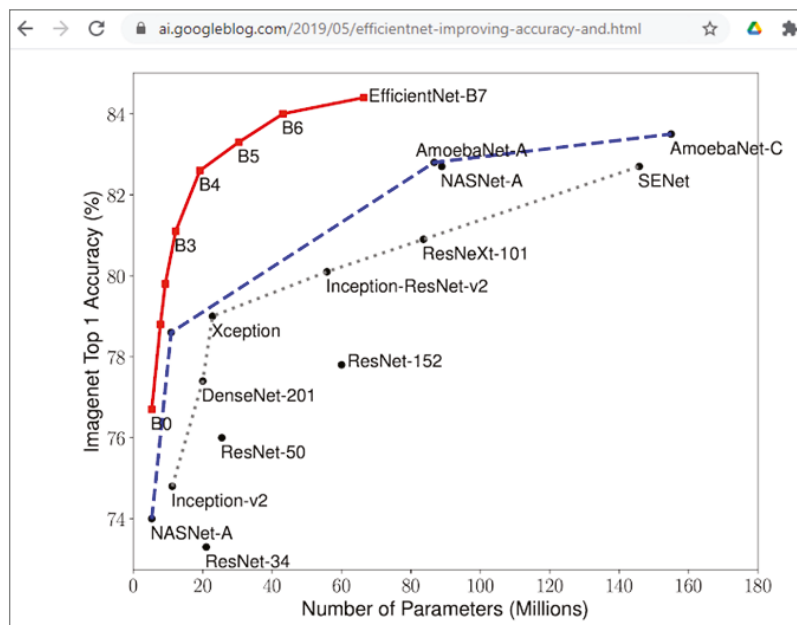


Figure 4.17: Accuracies of different deep learning neural networks on ImageNet

(Source: <https://ai.googleblog.com/2019/05/efficientnet-improving-accuracy-and.html>)

EfficientNet is also at the top on the ImageNet classification leaderboard, as shown in Figure 4.18.

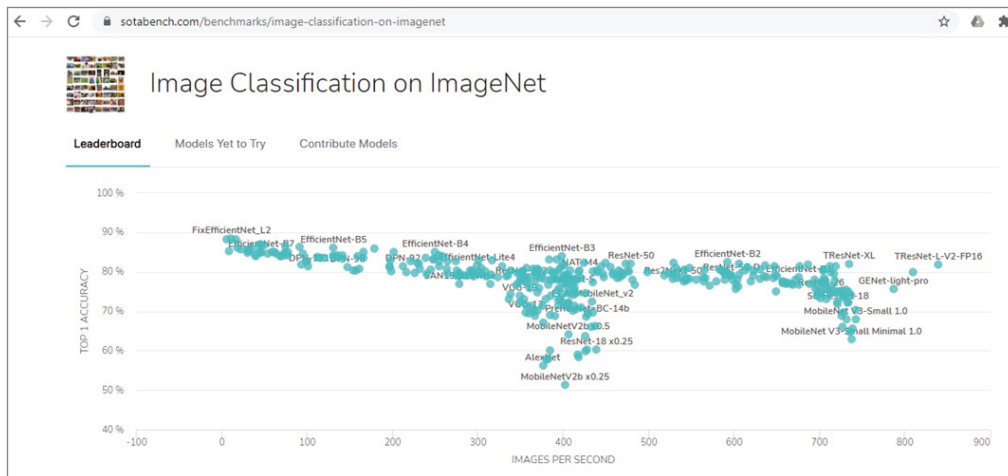


Figure 4.18: The SOTA board for image classification on ImageNet

(Source: <https://sotabench.com/benchmarks/image-classification-on-imagenet>)

For more details of EfficientNet, see the following:

<https://arxiv.org/abs/1905.11946>

<https://github.com/tensorflow/tpu/tree/master/models/official/efficientnet>

YOLO (you only look once) is a state-of-the-art system for real-time object detection. Unlike other convolutional neural networks, YOLO applies a single neural network to the entire image and then divides the image into regions and predicts bounding boxes and probabilities for each region. These bounding boxes are weighted by the predicted probabilities. For more detail, visit the YOLO website:

<https://pjreddie.com/darknet/yolo/>

Now let's look at some example code.

Example 4.10 shows the Python code for loading a deep learning neural network, such as VGG16, and printing the summary of the model.

EXAMPLE 4.10 THE VGG16.PY PROGRAM

```
#Example 4.10
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
from keras.applications.imagenet_utils import decode_predictions
import numpy as np

model = VGG16(weights='imagenet')
print(model.summary())
```

This is the output of the example, showing the summary of VGG16 network:

Using TensorFlow backend.

Model: "vgg16"

Layer (type)	Output Shape	Param #
=====		
input_1 (InputLayer)	[(None, 224, 224, 3)]	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1180160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2359808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2359808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2359808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
fc1 (Dense)	(None, 4096)	102764544
fc2 (Dense)	(None, 4096)	16781312

```

predictions (Dense)          (None, 1000)          4097000
=====
Total params: 138,357,544
Trainable params: 138,357,544
Non-trainable params: 0
=====
None

```

EXERCISE 4.9

Modify the Python program from Example 4.10 so that it loads the VGG19 model, instead of the VGG16 model.

Example 4.11 shows an improved version of the previous example code. In this case, it also uses the model to classify an image.

EXAMPLE 4.11 THE VGG16A.PY PROGRAM

```

#Example 4.11
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
from keras.applications.imagenet_utils import decode_predictions
import numpy as np

model = VGG16(weights='imagenet')

img_path = 'Elephant.jpg'
img = image.load_img(img_path, target_size=(224, 224))
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)

# prediction
predictions = model.predict(x)
results = decode_predictions(predictions)

print(results)

```

This is the output of the example, showing the classification results:

```

[(['n02504458', 'African_elephant', 0.80238396), ('n01871265',
'tusker', 0.12562281), ('n02504013', 'Indian_elephant', 0.07094732),
('n02099849', 'Chesapeake_Bay_retriever', 0.00035384137), ('n02437312',
'Arabian_camel', 0.00018172464)]]

```

EXERCISE 4.10

Modify the Python program from Example 4.11 to use VGG19 for classification and compare its performance with VGG16.

Example 4.12 shows a Python code that can load a set of deep learning neural network models, such as VGG16, ResNet50, MobileNet, and Inception V3, display the summary of the models, and use the model to classify an image.

EXAMPLE 4.12 THE MULTICLASSIFY.PY PROGRAM

```
#Example 4.12
from tensorflow.keras.preprocessing import image
from keras.applications.imagenet_utils import decode_predictions
import numpy as np

from tensorflow.keras.applications import (
    vgg16,
    resnet50,
    mobilenet,
    inception_v3
)

# init the models
#model = vgg16.VGG16(weights='imagenet')
#model = resnet50.ResNet50(weights='imagenet')
model = mobilenet.MobileNet(weights='imagenet')
#model = inception_v3.InceptionV3(weights='imagenet')
print(model.summary())

img_path = 'Elephant.jpg'
img = image.load_img(img_path, target_size=(224, 224))
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
#processed_image = vgg16.preprocess_input(x)
#processed_image = resnet50.preprocess_input(x)
processed_image = mobilenet.preprocess_input(x)
#processed_image = inception_v3.preprocess_input(x)

# prediction
predictions = model.predict(x)
results = decode_predictions(predictions)
print(results)
```

A web camera, or webcam, is a useful tool to capture images to provide data resource for image classification using deep learning. Example 4.13 shows a Python code for image classification based on the image array from a web camera (webcam), using VGG16 deep learning neural network.

EXAMPLE 4.13 THE WEBCAMCLASSIFY.PY PROGRAM

```

#Example 4.13

import cv2
from tensorflow.keras.preprocessing.image import img_to_array
from keras.applications.imagenet_utils import decode_predictions
from tensorflow.keras.applications import vgg16
import numpy as np

image_size = 224
model = vgg16.VGG16(weights='imagenet')
print(model.summary())

camera = cv2.VideoCapture(0)

while camera.isOpened():
    ok, cam_frame = camera.read()

    frame= cv2.resize(cam_frame, (image_size, image_size))
    numpy_image = img_to_array(frame)
    image_batch = np.expand_dims(numpy_image, axis=0)
    processed_image = vgg16.preprocess_input(image_batch.copy())

    # get the predicted probabilities for each class
    predictions = model.predict(processed_image)
    label = decode_predictions(predictions)

    # format final image visualization to display the results of
    experiments
    cv2.putText(cam_frame, "VGG16: {}, {:.1f}".format(label[0][0][1],
label[0][0][2]) , (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0),
2)

    cv2.imshow('video image', cam_frame)

    key = cv2.waitKey(30)
    if key == 27: # press 'ESC' to quit
        break

camera.release()
cv2.destroyAllWindows()

```

Figure 4.19 shows the output of the previous VGG16 classifier based on the images from a webcam, which correctly identifies a computer mouse with 90 percent confidence.



Figure 4.19: The output of VGG16 webcam classification program

EXERCISE 4.11

Modify the Python VGG16 webcam program to use VGG19 to classify webcam images. Compare the performances.

Example 4.14 shows a Python code for image classification based on a web camera using different deep learning neural networks such as VGG16, ResNet50, MobileNet, and Google's Inception V3.

EXAMPLE 4.14 THE WEBCAMCLASSIFY2.PY PROGRAM

```
# Example 4.14
import cv2
from tensorflow.keras.preprocessing.image import img_to_array
from keras.applications.imagenet_utils import decode_predictions
from tensorflow.keras.applications import (
    vgg16,
    resnet50,
    mobilenet,
    inception_v3
)
import numpy as np

# init the models
#model = vgg16.VGG16(weights='imagenet')
model = resnet50.ResNet50(weights='imagenet')
#model = mobilenet.MobileNet(weights='imagenet')
#model = inception_v3.InceptionV3(weights='imagenet')
```

```
print(model.summary())

camera = cv2.VideoCapture(0)
image_size = 224
#image_size = 299
while camera.isOpened():
    ok, cam_frame = camera.read()

    frame= cv2.resize(cam_frame, (image_size, image_size))
    numpy_image = img_to_array(frame)
    image_batch = np.expand_dims(numpy_image, axis=0)

    #processed_image = vgg16.preprocess_input(image_batch.copy())
    processed_image = resnet50.preprocess_input(image_batch.copy())
    #processed_image = mobilenet.preprocess_input(image_batch.copy())
    #processed_image = inception_v3.preprocess_input(image_batch
    .copy())

    # get the predicted probabilities for each class
    predictions = model.predict(processed_image)
    label = decode_predictions(predictions)

    # format final image visualization to display the results of
    experiments
    cv2.putText(cam_frame, "{}, {:.1f}".format(label[0][0][1],
label[0][0][2]) , (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0),
2)

    cv2.imshow('video image', cam_frame)

    key = cv2.waitKey(30)
    if key == 27: # press 'ESC' to quit
        break
```

EXERCISE 4.12

Modify the Python program from Example 4.14 and add DenseNet to the model to classify webcam images. Compare the performances.

The Image-Classifier library is a classification model zoo library that supports 13 different types of deep learning neural networks. It is based on Keras and TensorFlow (<https://pypi.org/project/image-classifiers/>). To use the library, you must first install it.

```
pip install image-classifiers
```

Example 4.15 shows a Python code for image classification based on a web camera using Image-Classifier library, which supports a number of different deep learning neural networks such as VGG, ResNet, MobileNet, and Google's Inception, etc.

EXAMPLE 4.15 THE WEBCLASSIFY3.PY PROGRAM

```
# Example 4.15
#https://pypi.org/project/image-classifiers/

!pip install image-classifiers
from keras.applications.imagenet_utils import decode_predictions
from keras.applications.imagenet_utils import preprocess_input
from classification_models.keras import Classifiers

import numpy as np
import cv2

#Set up a model
clf, preprocess_input = Classifiers.get('vgg16')
#clf, preprocess_input = Classifiers.get('resnet50')
#clf, preprocess_input = Classifiers.get('mobilenetv2')
#clf, preprocess_input = Classifiers.get('densenet201')
sz = 224
#clf, preprocess_input = Classifiers.get('inceptionv3')
#sz = 299
model = clf(input_shape=(sz,sz,3), weights='imagenet', classes=1000)

model.summary()
while True:
    ret, cam_frame = camera.read()
    frame= cv2.resize(cam_frame, (image_size, image_size))
    image = np.asarray(frame)
    image = np.expand_dims(image, 0)
    image = preprocess_input(image)

    preds = model.predict(image)
    label = decode_predictions(preds)

    cv2.putText(cam_frame, "{}, {:.1f}".format(label[0][0][1],
label[0][0][2]),
                (10, 30),cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 255, 0), 2)
    cv2.imshow("Classification", cam_frame)
    key = cv2.waitKey(30)
    if key == 27: # press 'ESC' to quit
        break

camera.release()
cv2.destroyAllWindows()
```

EXERCISE 4.13

Modify the Python program from Example 4.15, uncomment the different models, and compare the performances.

EXERCISE 4.14

Modify the Python program from Example 4.15 to use an `if-else` statement to select the models for classification.

4.3.3 U-Net

U-Net is a convolutional neural network, developed for biomedical image segmentation at the Computer Vision Group, Department of Computer Science, the University of Freiburg, Germany. Figure 4.20 shows the U-Net website with its architecture.

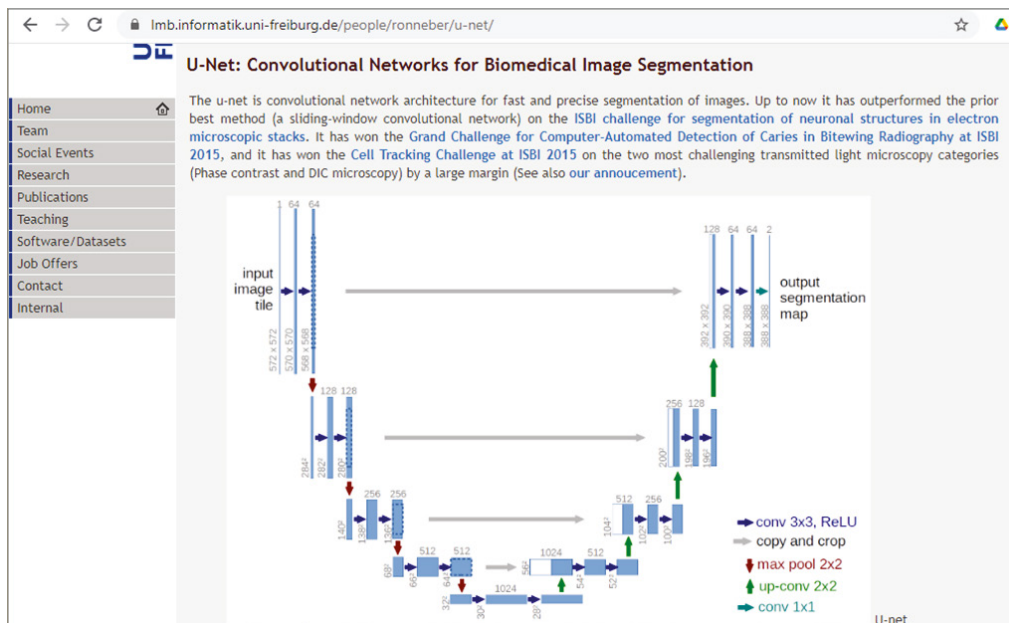


Figure 4.20: The U-Net website at Computer Vision Group, Department of Computer Science, the University of Freiburg, Germany

(Source: <https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/>)

U-Net is based on a fully convolutional network and has an input size of 572×572 . It uses three convolution layers and pooling layers to reduce the

input image to a 1,024-feature vector. It then uses three pooling and convolution layers to bring the feature vector back to 382×382 as a segmented image. The architecture looks like a *U*, which is where it gets its name.

To date, U-Net has been used in many applications in biomedical image segmentation, such as brain image segmentation, liver image segmentation, and medical image reconstruction

The easiest way to implement U-Net is to use the Keras-Unet library. See the following links for more details:

<https://pypi.org/project/keras-unet/>
<https://github.com/karolzak/keras-unet>

To install the Keras-Unet library, do this:

```
pip install keras-unet
```

To create a simple vanilla U-Net model and display a summary of the model, use the following code:

```
from keras_unet.models import vanilla_unet
model = vanilla_unet(input_shape=(512, 512, 3))
model.summary()
```

The following is the model summary of a vanilla U-Net. As you can see, the U-Net has changed from the input size ($512 \times 512 \times 3$) gradually down to ($44 \times 44 \times 512$) and then back up to ($324 \times 324 \times 1$).

Model: "model"			
Layer (type)	Output Shape	Param #	Connected to
=====			
input_1 (InputLayer)	(None, 512, 512, 3)	0	
conv2d (Conv2D)	(None, 510, 510, 64)	1792	input_1[0][0]
conv2d_1 (Conv2D)	(None, 508, 508, 64)	36928	conv2d[0][0]
max_pooling2d (MaxPooling2D)	(None, 254, 254, 64)	0	conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 252, 252, 128)	73856	max_pooling2d[0][0]
conv2d_3 (Conv2D)	(None, 250, 250, 128)	147584	conv2d_2[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 125, 125, 128)	0	conv2d_3[0][0]
conv2d_4 (Conv2D)	(None, 123, 123, 256)	295168	max_pooling2d_1[0][0]
conv2d_5 (Conv2D)	(None, 121, 121, 256)	590080	conv2d_4[0][0]
max_pooling2d_2 (MaxPooling2D)	(None, 60, 60, 256)	0	conv2d_5[0][0]

conv2d_6 (Conv2D)	(None, 58, 58, 512)	1180160	max_pooling2d_2[0][0]
conv2d_7 (Conv2D)	(None, 56, 56, 512)	2359808	conv2d_6[0][0]
max_pooling2d_3 (MaxPooling2D)	(None, 28, 28, 512)	0	conv2d_7[0][0]
dropout (Dropout)	(None, 28, 28, 512)	0	max_pooling2d_3[0][0]
conv2d_8 (Conv2D)	(None, 26, 26, 1024)	4719616	dropout[0][0]
conv2d_9 (Conv2D)	(None, 24, 24, 1024)	9438208	conv2d_8[0][0]
conv2d_transpose (Conv2DTranspo	(None, 48, 48, 512)	2097664	conv2d_9[0][0]
cropping2d (Cropping2D)	(None, 48, 48, 512)	0	conv2d_7[0][0]
concatenate (Concatenate)	(None, 48, 48, 1024)	0	conv2d_transpose[0][0]
			cropping2d[0][0]
conv2d_10 (Conv2D)	(None, 46, 46, 512)	4719104	concatenate[0][0]
conv2d_11 (Conv2D)	(None, 44, 44, 512)	2359808	conv2d_10[0][0]
conv2d_transpose_1 (Conv2DTrans	(None, 88, 88, 256)	524544	conv2d_11[0][0]
cropping2d_1 (Cropping2D)	(None, 88, 88, 256)	0	conv2d_5[0][0]
concatenate_1 (Concatenate)	(None, 88, 88, 512)	0	conv2d_transpose_1[0][0]
			cropping2d_1[0][0]
conv2d_12 (Conv2D)	(None, 86, 86, 256)	1179904	concatenate_1[0][0]
conv2d_13 (Conv2D)	(None, 84, 84, 256)	590080	conv2d_12[0][0]
conv2d_transpose_2 (Conv2DTrans	(None, 168, 168, 128)	131200	conv2d_13[0][0]
cropping2d_2 (Cropping2D)	(None, 168, 168, 128)	0	conv2d_3[0][0]
concatenate_2 (Concatenate)	(None, 168, 168, 256)	0	conv2d_transpose_2[0][0]
			cropping2d_2[0][0]
conv2d_14 (Conv2D)	(None, 166, 166, 128)	295040	concatenate_2[0][0]
conv2d_15 (Conv2D)	(None, 164, 164, 128)	147584	conv2d_14[0][0]
conv2d_transpose_3 (Conv2DTrans	(None, 328, 328, 64)	32832	conv2d_15[0][0]

cropping2d_3 (Cropping2D)	(None, 328, 328, 64) 0	conv2d_1[0][0]
concatenate_3 (Concatenate)	(None, 328, 328, 128) 0	conv2d_transpose_3[0][0]
		cropping2d_3[0][0]
conv2d_16 (Conv2D)	(None, 326, 326, 64) 73792	concatenate_3[0][0]
conv2d_17 (Conv2D)	(None, 324, 324, 64) 36928	conv2d_16[0][0]
conv2d_18 (Conv2D)	(None, 324, 324, 1) 65	conv2d_17[0][0]
=====		
Total params: 31,031,745		
Trainable params: 31,031,745		
Non-trainable params: 0		

The following code shows how to plot the U-Net model and save the model into a Portable Network Graphics (PNG) file. Figure 4.21 shows the plot of the U-Net.

```
from keras.utils import plot_model
plot_model(model, to_file='model.png')
```

You can also create a customized U-Net model and display the model summary using the following code:

```
from keras_unet.models import custom_unet

model = custom_unet(
    input_shape=(512, 512, 3),
    use_batch_norm=False,
    num_classes=1,
    filters=64,
    dropout=0.2,
    output_activation='sigmoid')
model.summary()
```

The quickest way to try image segmentation with the Keras-Unet library is through its GitHub repository. It uses Keras U-Net to segment images from a serial section Transmission Electron Microscopy (ssTEM) dataset of the *Drosophila* first instar larva ventral nerve cord (VNC), as shown in Figure 4.22.

Just follow the code in this IPython Notebook:

<https://github.com/karolzak/keras-unet/blob/master/notebooks/kz-isbi-challenge.ipynb>



Figure 4.21: The plot of vanilla U-Net model

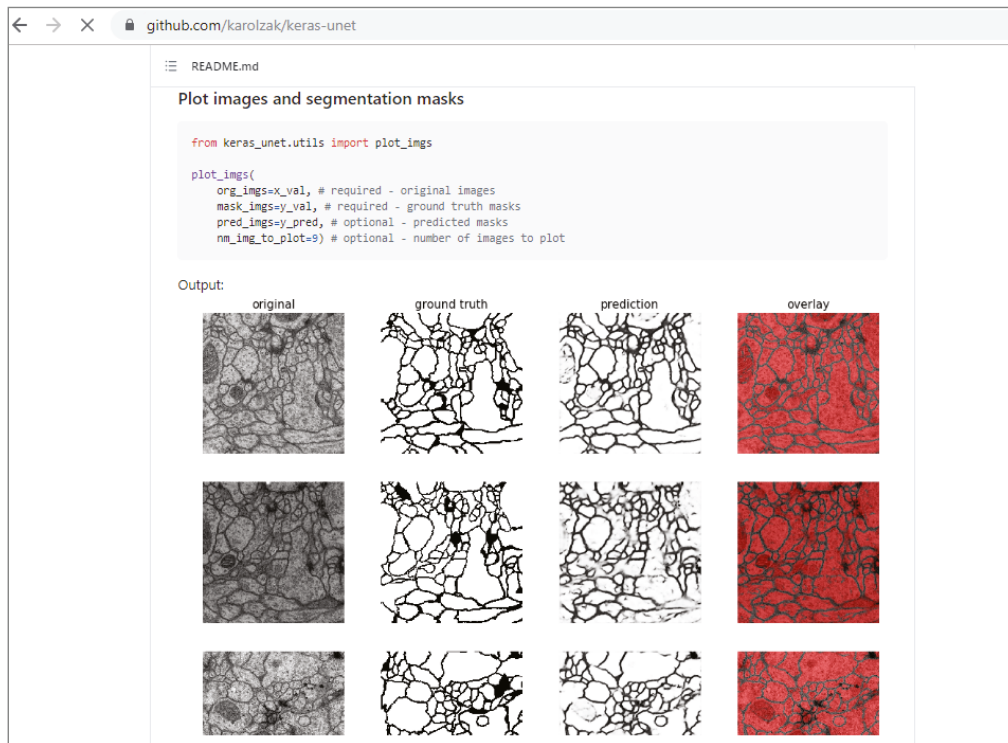


Figure 4.22: The Keras-Unet library GitHub website with serial section Transmission Electron Microscopy images

(Source: <https://github.com/karolzak/keras-unet>)

The following are more examples of U-Net projects:

<https://github.com/zhixuhao/unet>

<https://github.com/Hsankesara/DeepResearch>

<https://github.com/devswaha/keras-Unet>

<http://deeplearning.net/tutorial/unet.html>

<https://www.kaggle.com/phoenigs/u-net-dropout-augmentation-stratification>

<https://www.kaggle.com/kmader/baseline-u-net-model-part-1>

4.3.4 AutoEncoder

AutoEncoder is a neural network architecture, consisting of two connected networks: Encoder and Decoder, as illustrated in Figure 4.23. The Encoder receives the input image and encodes it into a latent spatial vector of a lower dimension. The network is trained to ignore signal noise during this dimensionality reduction. The Decoder takes this vector and decodes it to produce an output that is as close as possible to the original input. Autoencoders are developed with the goal of learning the low-dimensional feature representations of the input data.

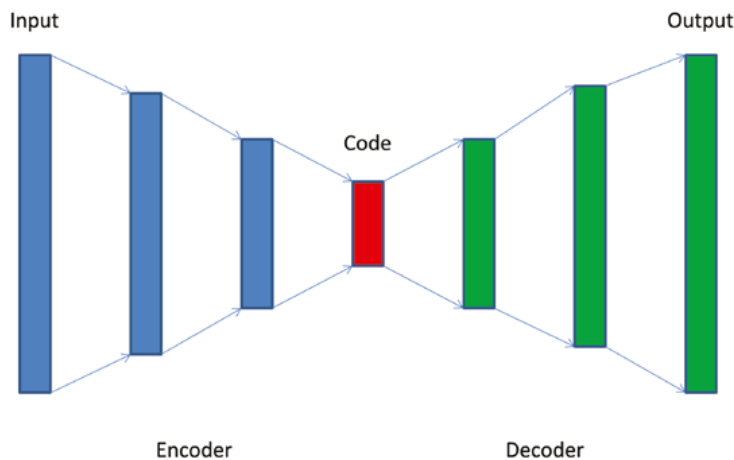


Figure 4.23: The AutoEncoder architecture with two connected networks: Encoder and Decoder

Example 4.16 shows a Python AutoEncoder example code. It was modified based on the example code (<https://stackabuse.com/autoencoders-for-image-reconstruction-in-python-and-keras/>). It first loads the MNIST handwritten digital images and normalizes the pixel values to 1. Then it builds a simple AutoEncoder that includes a simple encoder and a simple decoder. It then trains the AutoEncoder and uses the trained AutoEncoder to reconstruct the digits.

EXAMPLE 4.16 THE AUTOENCODERKERAS.PY PROGRAM

```
# Example 4.16 AutoencoderKeras.py
# Modified from:
# https://stackabuse.com/autoencoders-for-image-reconstruction-in-python-
# and-keras/
#
# Load the MNIST handwritten digital images =====
from keras.datasets import mnist
import numpy as np
(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
print(x_train.shape)
print(x_test.shape)

# Build Autoencoder =====
from keras.layers import Dense, Flatten, Reshape, Input, InputLayer
from keras.models import Sequential, Model

def build_autoencoder(img_shape, code_size):
    # The encoder
    encoder = Sequential()
    encoder.add(InputLayer(img_shape))
```

```

encoder.add(Flatten())
encoder.add(Dense(code_size))

# The decoder
decoder = Sequential()
decoder.add(InputLayer((code_size,)))
decoder.add(Dense(np.prod(img_shape)))
decoder.add(Reshape(img_shape))

return encoder, decoder

IMG_SHAPE = x_train[0].shape
encoder, decoder = build_autoencoder(IMG_SHAPE, 32)

inp = Input(IMG_SHAPE)
code = encoder(inp)
reconstruction = decoder(code)

autoencoder = Model(inp, reconstruction)
autoencoder.compile(optimizer='adamax', loss='mse')

print(autoencoder.summary())

# Train Autoencoder =====
history = autoencoder.fit(x=x_train, y=x_train, epochs=20,
                        validation_data=(x_test, x_test))

# Plot the training results =====
import matplotlib.pyplot as plt

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()

# Plot the original, encoded and decoded images =====
import matplotlib.pyplot as plt
def show_image(x):
    plt.imshow(np.clip(x + 0.5, 0, 1))

def visualize(img, encoder, decoder):
    """Draws original, encoded and decoded images"""
    # img[None] will have shape of (1, 32, 32, 3) which is the same as the
    model input
    code = encoder.predict(img[None])[0]
    reco = decoder.predict(code[None])[0]

```

```

plt.subplot(1,3,1)
plt.title("Original")
show_image(img)

plt.subplot(1,3,2)
plt.title("Code")
plt.imshow(code.reshape([code.shape[-1]//2,-1]))

plt.subplot(1,3,3)
plt.title("Reconstructed")
show_image(reco)
plt.show()

for i in range(5):
    img = x_test[i]
    visualize(img,encoder,decoder)

```

This is the output of the example, showing the structures of AutoEncoder:

```

(60000, 28, 28)
(10000, 28, 28)
Model: "model"

```

Layer (type)	Output Shape	Param #
input_3 (InputLayer)	[(None, 28, 28)]	0
sequential (Sequential)	(None, 32)	25120
sequential_1 (Sequential)	(None, 28, 28)	25872
Total params: 50,992		
Trainable params: 50,992		
Non-trainable params: 0		
None		

Figure 4.24 shows the example outputs of the AutoEncoder program, which includes the original digit images (left), the output of the encoder called code (middle), and the reconstructed digit images by the decoder.

For more information about AutoEncoders, see the following:

<https://blog.keras.io/building-autoencoders-in-keras.html>

<https://stackabuse.com/autoencoders-for-image-reconstruction-in-python-and-keras/>

<https://www.analyticsvidhya.com/blog/2020/02/what-is-autoencoder-enhance-image-resolution/>

<https://www.kaggle.com/shivamb/how-autoencoders-work-intro-and-usecases>

<https://theaisummer.com/Autoencoder/>

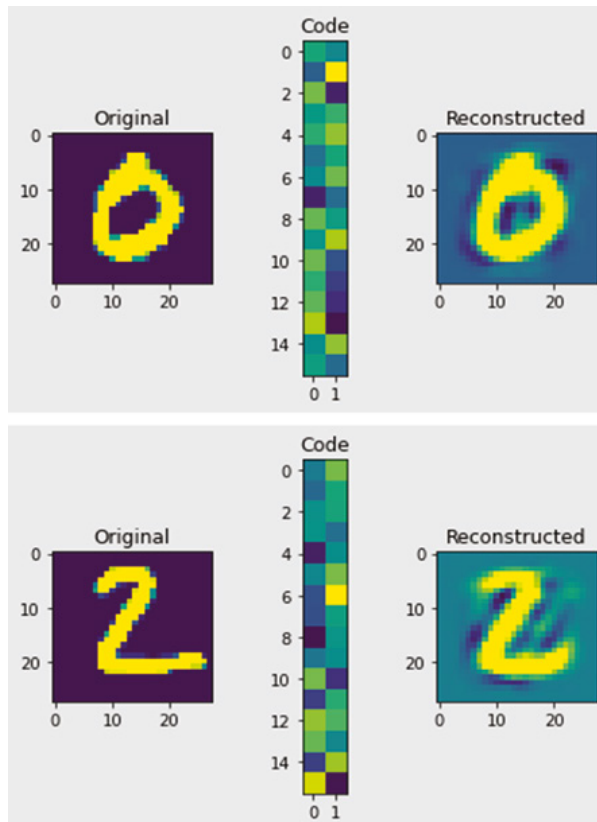


Figure 4.24: The example output of AutoEncoder program, which includes the original digit images (left), the output of the encoder called code (middle), and the reconstructed digit images by decoder (right).

4.3.5 Siamese Neural Networks

When I first started looking into deep learning, one thing always puzzled me. Take image classification, for example. Deep learning neural networks require a large amount of labeled training data to achieve high performance. However, we humans are able to learn things with just a glance at the object. Even if we have never seen passion fruits, we don't need to see thousands and thousands of images before we can recognize them.

Can deep learning neural networks learn the same way? The answer is yes, this is called *one-shot learning*. In one-shot learning, we need only a single training example for each class instead of training a large set of images. The most commonly used one-shot learning neural network is the Siamese network.

Figure 4.25 shows an interesting introduction of one-shot learning and Siamese networks in Keras. It describes a typical architecture of a convolutional Siamese network as an example of classifying pairs of omniglot images (<https://github.com/brendenlake/omniglot>). The omniglot data contains 1,623 different handwritten characters from 50 different alphabets.

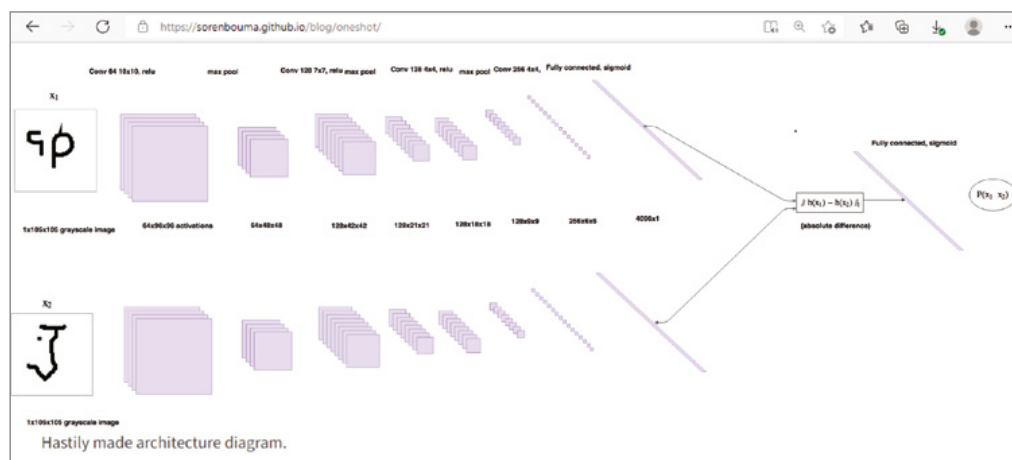


Figure 4.25: One-shot learning and Siamese networks in Keras

(Source: <https://sorenbouma.github.io/blog/oneshot/>)

The Siamese network contains two convolutional neural networks, called *twin neural network*. The two neural networks reduce the inputs to lower-dimension tensors; finally, there is a fully connected layer with 4,096 units. The absolute difference between the two vectors is used as input to a linear classifier. A linear classifier performs a classification decision based on the value of a linear combination of the characteristics. The network has 38,951,745 parameters, 96 percent of which belong to the fully connected layer. Training Siamese networks in this way with comparative loss functions can give better performances. Siamese networks have also been used for face recognition.

To try a Siamese network for classifying omniglot images, just follow this interesting IPython notebook:

<https://github.com/sorenbouma/keras-oneshot/blob/master/SiameseNet.ipynb>

For more information about Siamese networks, visit the following websites:

<https://github.com/tensorfreitas/Siamese-Networks-for-One-Shot-Learning>
<https://towardsdatascience.com/one-shot-learning-with-siamese-networks-using-keras-17f34e75bb3d>
<https://github.com/hlamba28/One-Shot-Learning-with-Siamese-Networks>
<https://github.com/akshaysharma096/Siamese-Networks>
<https://github.com/sorenbouma/keras-oneshot>
<https://github.com/aspamers/siamese>
<https://www.cs.cmu.edu/~rsalakhu/papers/oneshot1.pdf>
<https://machinelearningmastery.com/one-shot-learning-with-siamese-networks-contrastive-and-triplet-loss-for-face-recognition/>

Apart from one-shot learning, there are also few-shot learning and zero-shot learning. Few-shot learning means that the neural network is trained with a few (one to five) images. Zero-shot learning means that the neural network is trained with zero images. This is fascinating! Few-shot learning and zero-shot learning are sometimes categorized as N-shot learning, which means that the neural network is trained with N images.

The key concept of n-shot learning is the prototypical network. Unlike typical deep learning neural networks, prototypical networks do not classify the image directly; instead, they learn the mapping of images in the metric space and group the images into different clusters, called *prototypes*. In metric space, there is no distinguished “origin” point; instead, you just calculate the distance from one point to another. Prototypical networks then use the distances between prototypes and encoded query images to classify them.

N-shot learning is a learning algorithm that allows the network to learn more with less data. It is typically used when it is hard to find training data, for example, rare diseases, or when the cost of labeling data is too high.

For more details, check the following interesting blog and the GitHub code to the blog:

<https://blog.floydhub.com/n-shot-learning/>
<https://github.com/Hsankesara/Prototypical-Networks>

4.3.6 Capsule Networks

Convolutional neural networks perform best when the image to be classified is similar to the training image dataset. If the image to be classified is different (e.g., rotated, tilted, or transitioned), then the performance drops. Take face detection as an example; if the eyes, nose, and mouth are detected, the image is classified as a face, even if they are in the wrong place. Because of this, there are many shocking errors in image classification.

How can this problem be solved? The answer is a capsule network. A capsule network is a new deep learning network architecture proposed by Sara Sabour, Nicholas Frost, and Geoffrey Hinton in their paper titled “Dynamic Routing Between Capsules” in 2017. See Figure 4.26 for the original paper and a simple three-layer capsule network architecture called CapsNet. In this model, there are modules called *capsules* that are particularly good at handling different types of visual stimuli such as position, size, orientation, deformation, texture, and so on. As a result, the performance is significantly improved.

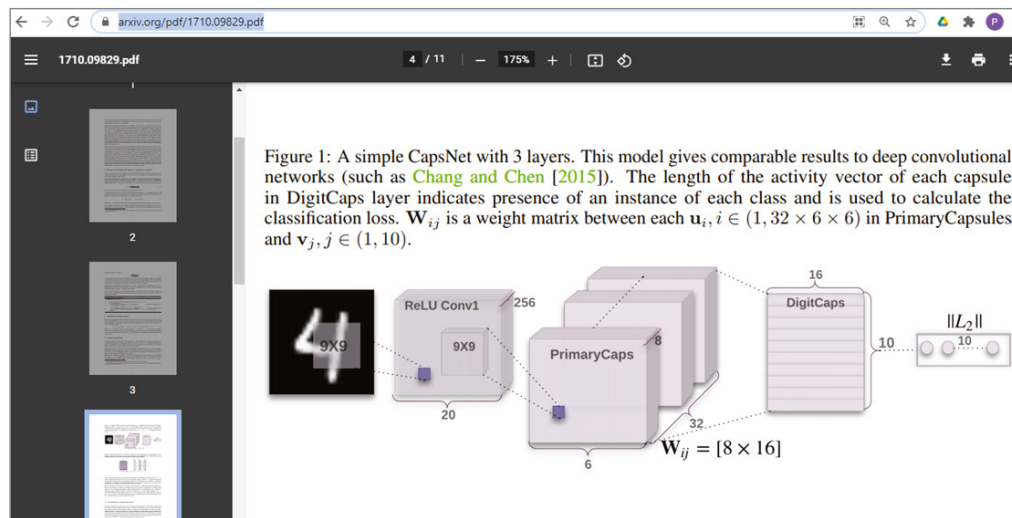


Figure 4.26: The original capsule network paper and a three-layer simple capsule network architecture

(Source: <https://arxiv.org/pdf/1710.09829.pdf>)

The following website shows an interesting simple introduction of a capsule network, which provides the full capsule network source code implemented in TensorFlow.

<https://hackernoon.com/what-is-a-capsnet-or-capsule-network-2bfbe48769cc>

This is the corresponding Github site for the source code.

<https://github.com/debarko/CapsNet-Tensorflow>

The following website shows the capsule network implementation in TensorFlow.

<https://github.com/debarko/CapsNet-Tensorflow>

Figure 4.27 shows the PyTorch implementation of a capsule graph neural network.

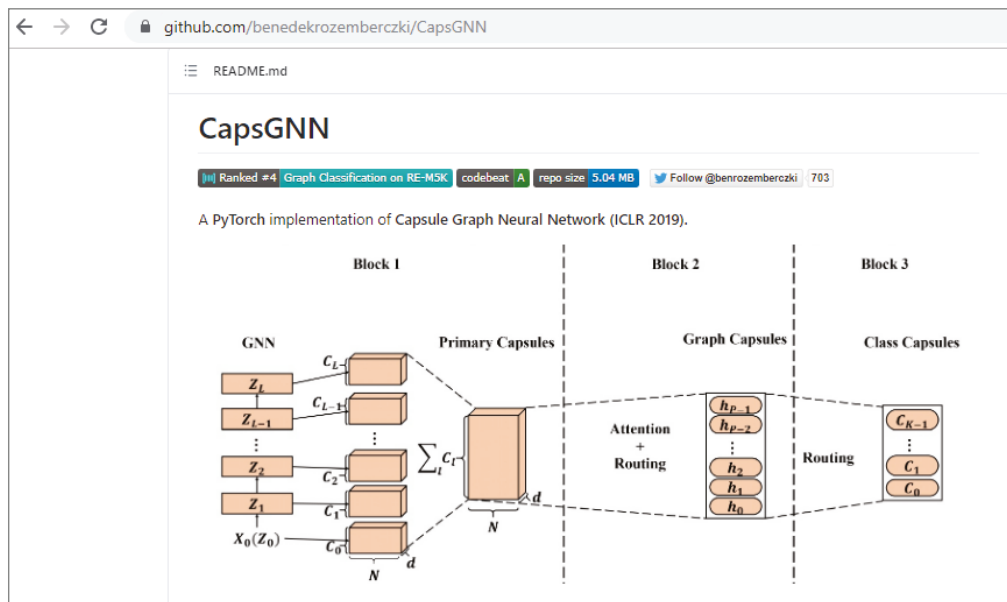


Figure 4.27: A PyTorch implementation of a capsule graph neural network

(Source: <https://github.com/benedekrozemberczki/CapsGNN>)

4.3.7 CNN Layers Visualization

During the development, you may want to see the output of each layer of a convolutional neural network (CNN), whether for debugging purposes or just out of curiosity.

The easiest way to do this is to visualize each layer individually. The following example shows how you can visualize the output of the different convolutional layers in the VGG19 model; it is modified based on an excellent article on visualizing the filters and feature maps in convolutional neural networks (<https://machinelearningmastery.com/how-to-visualize-filters-and-feature-maps-in-convolutional-neural-networks/>).

We will show the code section by section. You can find the full code in a file called `CNN_Visualize_Filters.ipynb`. We will run it by uploading it to Google Colab.

Example 4.17 is the first section of the code that shows how to load a VGG19 model and display the summary of the model.

EXAMPLE 4.17 THE CNN VISUALIZE FILTERS.IPYNB PROGRAM (PART 1)

```
# Example 4.17 CNN Visualize Filters.ipynb
#Modified from:
#https://machinelearningmastery.com/how-to-visualize-filters-and-
feature-maps-in-convolutional-neural-networks/
```

```
# load vgg model
from keras.applications.vgg19 import VGG19
# load the model
model = VGG19()
# summarize the model
model.summary()
```

The following is the output that shows the summary of the model:

Model: "vgg19"

Layer (type)	Output Shape	Param #
=====		
input_31 (InputLayer)	[(None, 224, 224, 3)]	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590080
block3_conv4 (Conv2D)	(None, 56, 56, 256)	590080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1180160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2359808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2359808
block4_conv4 (Conv2D)	(None, 28, 28, 512)	2359808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2359808

block5_conv2 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv4 (Conv2D)	(None, 14, 14, 512)	2359808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
fc1 (Dense)	(None, 4096)	102764544
fc2 (Dense)	(None, 4096)	16781312
predictions (Dense)	(None, 1000)	4097000
=====		
Total params: 143,667,240		
Trainable params: 143,667,240		
Non-trainable params: 0		

Example 4.17 shows how to display all the layers in a VGG19 model and show the layer numbers.

EXAMPLE 4.17 THE CNN VISUALIZE FILTERS.IPYNB PROGRAM (PART 2)

```
n=0
for layer in model.layers:
    print(n,layer.name)
    n+=1
```

The following is the output, showing the layer number and the layer name of all the 25 layers:

```
0 input_16
1 block1_conv1
2 block1_conv2
3 block1_pool
4 block2_conv1
5 block2_conv2
6 block2_pool
7 block3_conv1
8 block3_conv2
9 block3_conv3
10 block3_conv4
11 block3_pool
12 block4_conv1
13 block4_conv2
14 block4_conv3
```

```
15 block4_conv4
16 block4_pool
17 block5_conv1
18 block5_conv2
19 block5_conv3
20 block5_conv4
21 block5_pool
22 flatten
23 fc1
24 fc2
25 predictions
```

The following section of the code shows the filter shape and bias shape for the convolutional layers in the VGG16 model. It uses the function `layer.get_weights()` to get the filter and bias parameters.

EXAMPLE 4.17 THE CNN VISUALIZE FILTERS.IPYNB PROGRAM (PART 3)

```
n=0
for layer in model.layers:
    if 'conv' in layer.name:
        filters, biases = layer.get_weights()
        print(n, layer.name, filters.shape, biases.shape)
    n+=1
```

The following output shows all the convolutional layers, their layer number, and their filter and bias parameters. There are 20 convolutional layers. For layer 1, there are 3 color channels; the filter kernel is (3,3), the total number of filters is 64, and total number of bias is also 64.

```
1 block1_conv1 (3, 3, 3, 64) (64,)
2 block1_conv2 (3, 3, 64, 64) (64,)
4 block2_conv1 (3, 3, 64, 128) (128,)
5 block2_conv2 (3, 3, 128, 128) (128,)
7 block3_conv1 (3, 3, 128, 256) (256,)
8 block3_conv2 (3, 3, 256, 256) (256,)
9 block3_conv3 (3, 3, 256, 256) (256,)
10 block3_conv4 (3, 3, 256, 256) (256,)
12 block4_conv1 (3, 3, 256, 512) (512,)
13 block4_conv2 (3, 3, 512, 512) (512,)
14 block4_conv3 (3, 3, 512, 512) (512,)
15 block4_conv4 (3, 3, 512, 512) (512,)
17 block5_conv1 (3, 3, 512, 512) (512,)
18 block5_conv2 (3, 3, 512, 512) (512,)
19 block5_conv3 (3, 3, 512, 512) (512,)
20 block5_conv4 (3, 3, 512, 512) (512,)
```

The following section of the code shows the filters in the first convolutional layers ($n = 1$) in VGG19 model. Again, the function `layer.get_weights()` is used to get the filter and bias parameters. Also, the filter is normalized to 0–1 for the purpose of visualization. Then, the first four filters (`n_filters`, `ix = 4`, 1) of the total 64 filters are displayed in the first layer.

EXAMPLE 4.17 THE CNN VISUALIZE FILTERS.IPYNB PROGRAM (PART 4)

```
from matplotlib import pyplot
# retrieve weights from the first hidden layer
n = 1
filters, biases = model.layers[n].get_weights()
s = filters.shape
print("Color channels: ", s[0])
print("Filter size: ", s[1],s[2])

print("Total number of filters : ", s[3])
# normalize filter values to 0-1 so we can visualize them
f_min, f_max = filters.min(), filters.max()
filters = (filters - f_min) / (f_max - f_min)
# plot first few filters
#n_filters, ix = s[3], 1
n_filters, ix = 4, 1

pyplot.figure(figsize=(10,10))
for i in range(n_filters):
    # get the filter
    f = filters[:, :, :, i]

    # plot each channel separately
    for j in range(s[0]):
        ax = pyplot.subplot(n_filters, s[0], ix)
        ax.set_xticks([])
        ax.set_yticks([])
        pyplot.imshow(f[j,:,:], cmap='gray')
        ix += 1
# show the figure
pyplot.show()
```

Figure 4.28 shows the output of the previous code, which shows there are 3 color channels, (3,3) filter size, and total 64 filters in layer 1. It also displays the (3,3) filters in layer 1.

EXERCISE 4.15

Modify the Python code from the previous example, select the second convolutional layer to visualize by changing `n = 1` to `n = 2`, and comment on the results.

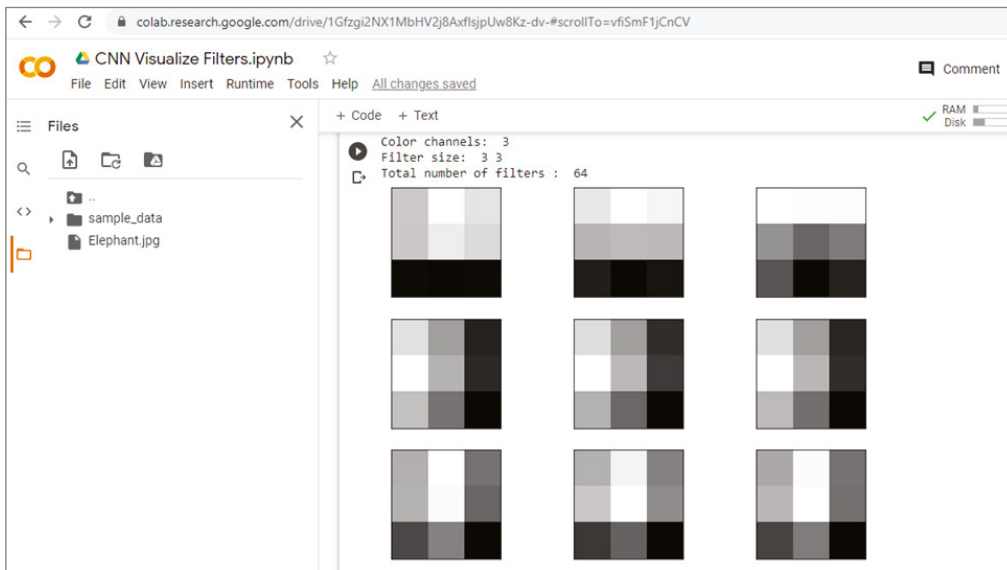


Figure 4.28: The output of the previous example code that displays the filters in layer 1

EXERCISE 4.16

Modify the Python code from the previous example, display all the filters by changing `n_filters`, `ix = 4`, `1 to n_filters`, `ix = s[3]`, `1`, and comment on the results.

The following shows a simple function to draw feature maps in a subplot with multiple rows and columns. The number of columns is set to eight, `col = 8`.

EXAMPLE 4.17 THE CNN VISUALIZE FILTERS.IPYNB PROGRAM (PART 5)

```
def plot_feature_maps(feature_maps):
    # plot all feature maps
    col = 8
    row = int(feature_maps.shape[3]/col)
    ix = 1
    plt.figure(figsize=(20,20))
    for _ in range(row):
        for _ in range(col):
            # specify subplot and turn of axis
            ax = plt.subplot(row, col, ix)
            ax.set_xticks([])
            ax.set_yticks([])
            # plot filter channel in grayscale
            plt.imshow(feature_maps[0, :, :, ix-1], cmap='gray')
            ix += 1
    # show the figure
    plt.show()
```

You can visualize the output of the first layer of the VGG19 model by setting the output of layer 1 ($n = 1$) as the model output; see `Model(inputs=model.inputs, outputs=model.layers[n].output)` in the following section of the code. Then `feature_maps = model.predict(img)` is used to get the output of layer 1. We call this layer 1 output as feature maps. Then, it calls `plot_feature_maps(feature_maps)` to display feature maps. The feature maps basically show what the image looks like after applying all the layer 1 filters. You can change to other layers by changing $n = 1$.

EXAMPLE 4.17 THE CNN VISUALIZE FILTERS.IPYNB PROGRAM (PART 6)

```
from keras.applications.vgg19 import VGG19
from keras.applications.vgg19 import preprocess_input
from keras.preprocessing.image import load_img
from keras.preprocessing.image import img_to_array
from keras.models import Model
import matplotlib.pyplot as plt
from numpy import expand_dims
# load the model
model = VGG19()
#Select the hidde layer to visualize
n = 1
# redefine model to output right after the hidden layer
model = Model(inputs=model.inputs, outputs=model.layers[n].output)
model.summary()

# load the image with the required shape
img = load_img('Elephant.jpg', target_size=(224, 224))
# convert the image to an array
img = img_to_array(img)
# expand dimensions so that it represents a single 'sample'
img = expand_dims(img, axis=0)
# prepare the image (e.g. scale pixel values for the vgg)
img = preprocess_input(img)
# get feature map for first hidden layer
feature_maps = model.predict(img)
print("Feature maps: ", feature_maps.shape)
#plot all the feature maps
plot_feature_maps(feature_maps)
```

Figure 4.29 shows the output of the previous code, which shows the structure of the VGG19 model with the first layer as the output. It also shows the feature maps (the output) of layer 1.

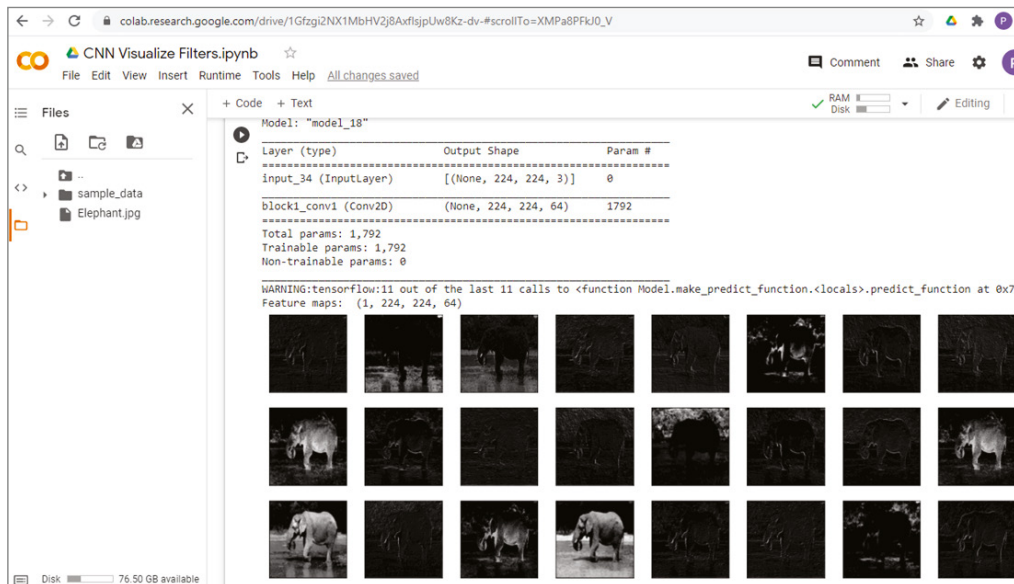


Figure 4.29: The output of the previous code, which shows the structure of the VGG19 model with the first layer as the output. It also shows the feature maps (the output) of layer 1.

EXERCISE 4.17

Modify the previous example Python code, select the second or third or fourth convolutional layer for visualization by changing $n = 1$ to $n = 2$ and so on, and comment on the results.

Another way to visualize each layer is to use the Keras Visualization Toolkit, see the following link for details.

<https://raghakot.github.io/keras-vis/>

Figure 4.30 shows the examples of convolution filter visualization, dense layer visualization, and attention maps in Keras Visualization Toolkit.

The following are a few more examples on the visualization of intermediate layers in convolutional neural networks:

https://keras.io/examples/conv_filter_visualization/
<https://www.analyticsvidhya.com/blog/2018/03/essentials-of-deep-learning-visualizing-convolutional-neural-networks/>
<https://github.com/JGuillaume/DeepLearning-NoBlaBla/blob/master/KerasViz.ipynb>
<https://towardsdatascience.com/visualizing-intermediate-activation-in-convolutional-neural-networks-with-keras-260b36d60d0>
<https://github.com/gabrielpierobon/cnnshapes/blob/master/README.md>
<https://www.kaggle.com/amarjeet007/visualize-cnn-with-keras>

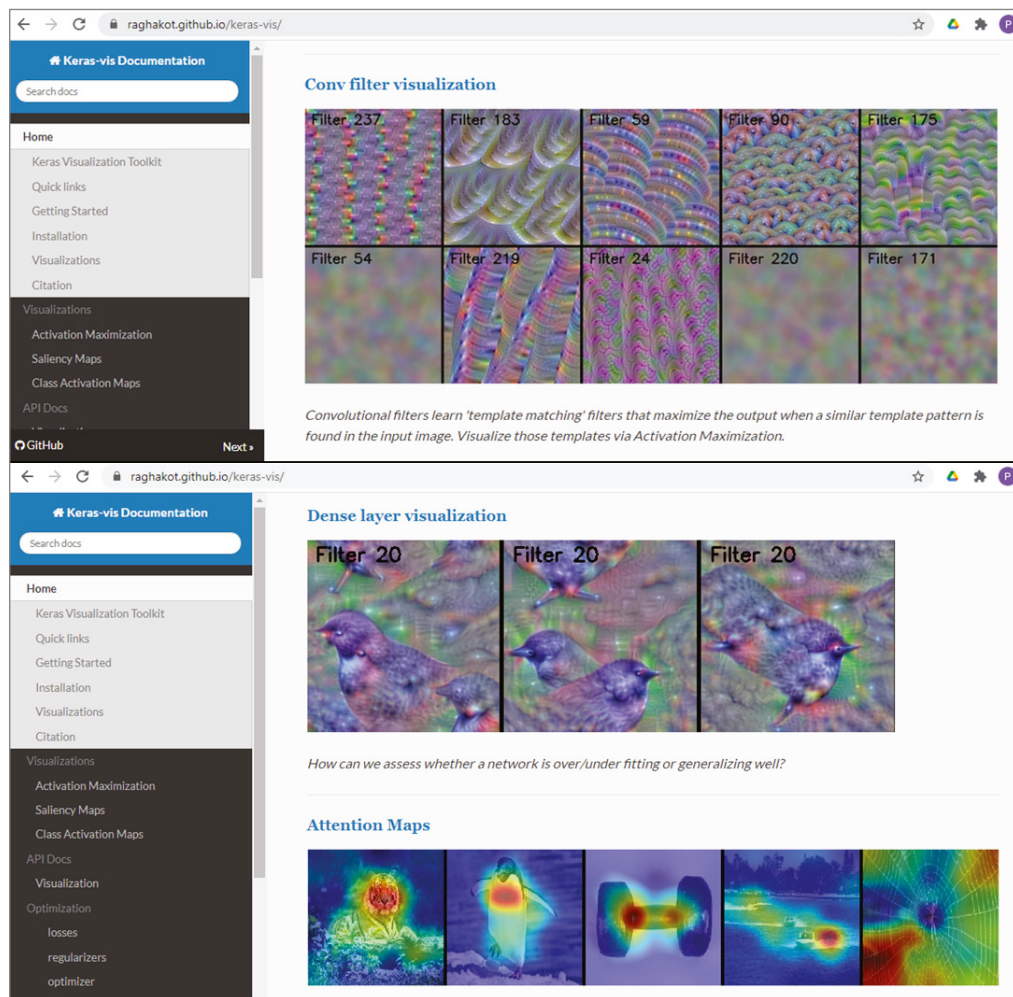


Figure 4.30: The convolutional filter visualization (top), the dense layer visualization, and the attention maps (bottom) in the Keras Visualization Toolkit
(Source: <https://raghakot.github.io/keras-vis/>)

4.4 Recurrent Neural Networks

Recurrent neural networks (RNNs) are another popular deep learning neural network, along with convolutional neural networks. Convolutional neural networks specialize in processing images, while RNNs specialize in processing sequences. RNNs are derived from feedforward neural networks and can use their internal state (memory) to process variable-length sequences of inputs. RNNs have been applied in sequence prediction, text analysis, and speech recognition.

Figure 4.31 shows the schematic diagram of a recurrent neural network and its unfolded structure. X is the sequential input data, O is the sequential output

data, h is the recurrent unit (also called cell), and v is the feedback loop. As shown in Figure 4.31, the recurrent structure occurs repeatedly in the network. The h recurrent unit can be implemented as long short-term memory (LSTM) network unit and a gated recurrent unit (GRU).

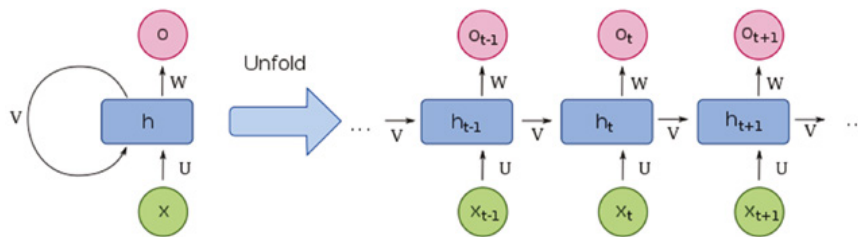


Figure 4.31: The schematic diagram of recurrent neural network and its unfolded structure
(Source: https://en.wikipedia.org/wiki/Recurrent_neural_network)

Figure 4.32 shows an interesting article that explains the differences of RNN, LSTM, and GRU.

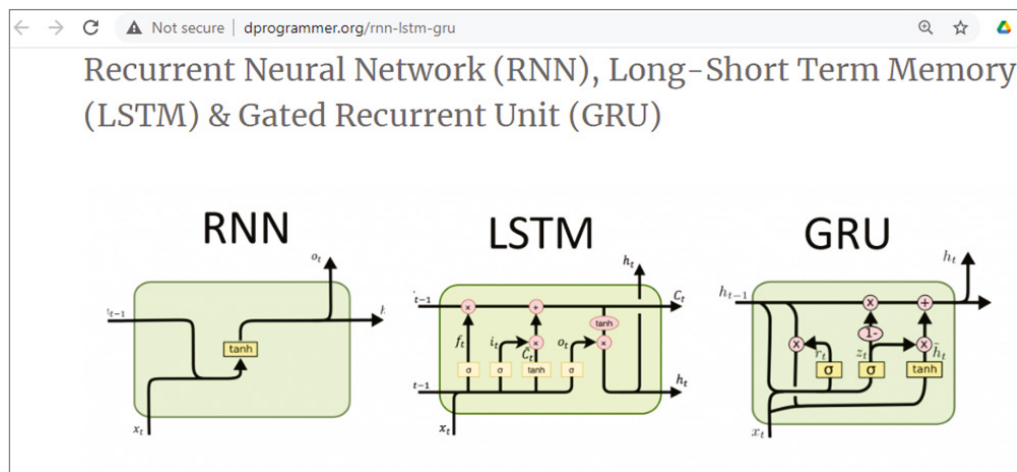


Figure 4.32: The schematic diagram of RNN, LSTM, and GRU
(Source: <http://dprogrammer.org/rnn-lstm-gru>)

Figure 4.33 shows the four types of recurrent neural networks.

- **One to one:** One input and one output
- **One to many:** One input and multiple outputs
- **Many to one:** Multiple inputs and one output
- **Many to many:** Multiple inputs and multiple outputs

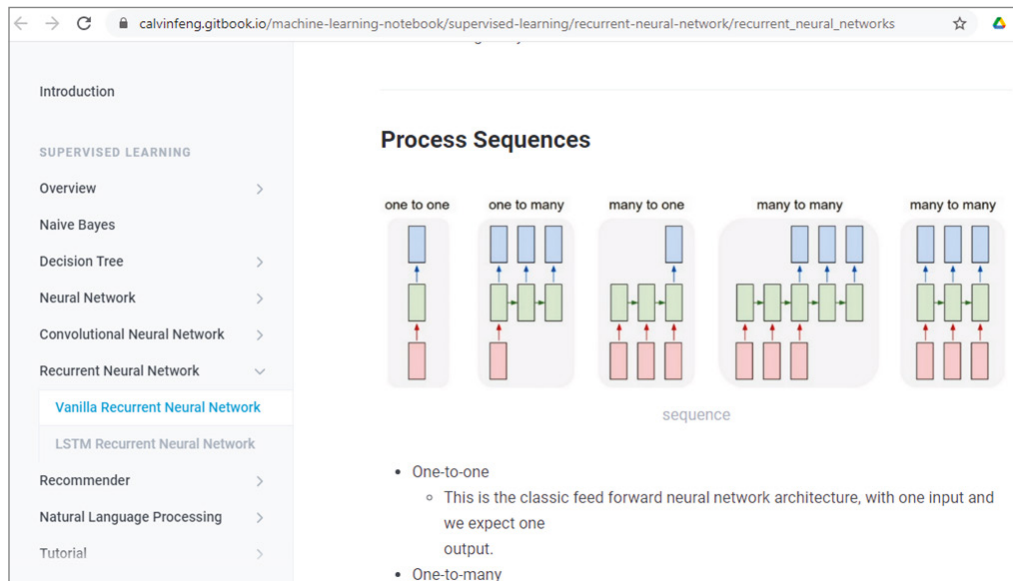


Figure 4.33: The different types of vanilla RNN

(Source: https://calvinfeng.gitbook.io/machine-learning-notebook/supervised-learning/recurrent-neural-network/recurrent_neural_networks)

For more details about recursive neural networks, see the following:

<https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks>

<https://www.simplilearn.com/tutorials/deep-learning-tutorial/rnn>

4.4.1 Vanilla RNNs

A vanilla RNN is the simplest recurrent neural network that has an input vector, an output vector, and a recurrent unit or cell, as shown in Figure 4.34. Within the recurrent unit, there is an activation function, called *tanh*. You can also use a different activation function.

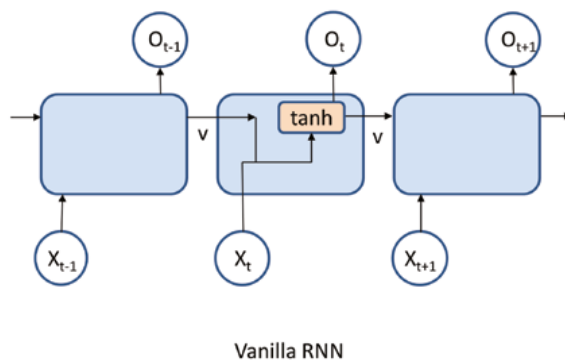


Figure 4.34: The vanilla RNN structure

(Source: <https://subscription.packtpub.com/book/data/9781789340990/5/ch05l1v11sec21/vanilla-rnns>)

The easiest way to implement vanilla RNN is to use the Keras function called SimpleRNN; see the following links for more details:

https://www.tensorflow.org/api_docs/python/tf/keras/layers/SimpleRNN
https://keras.io/api/layers/recurrent_layers/simple_rnn/

The following is another example of vanilla RNNs using Keras SimpleRNN. It comes from an impressive website called Easy-deep-learning-with-Keras, which contains many illustrative example codes on deep learning neural networks; see Figure 4.35.

<https://github.com/buomsoo-kim/Easy-deep-learning-with-Keras/blob/master/3.%20RNN/1-Basic-RNN/1-1-vanilla-rnn.py>

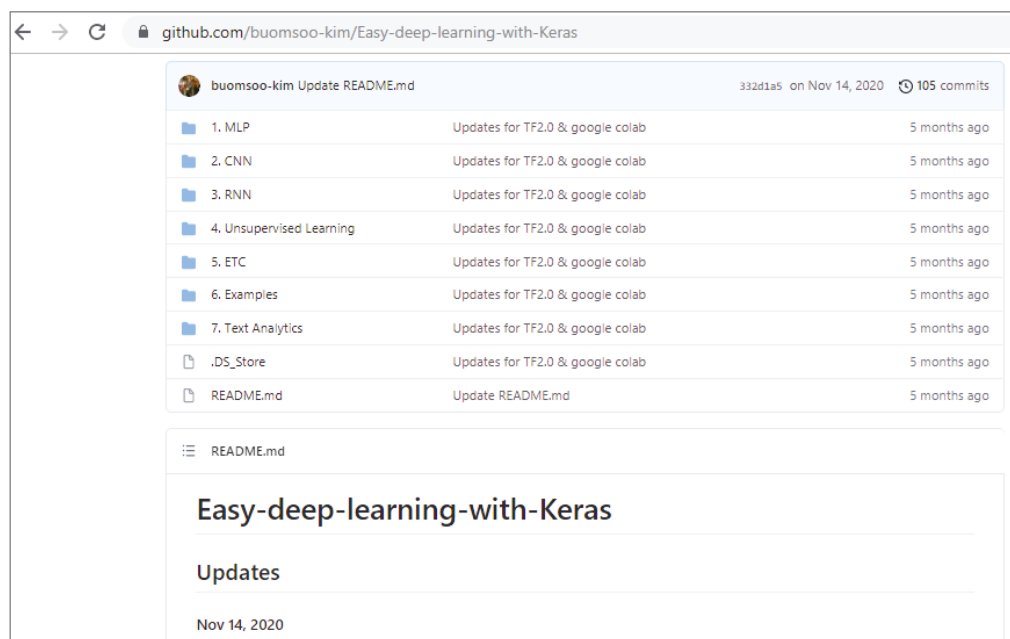


Figure 4.35: The Easy-deep-learning-with-Keras website

(Source: <https://github.com/buomsoo-kim/Easy-deep-learning-with-Keras>)

4.4.2 Long-Short Term Memory

Long short-term memory (LSTM) is the most popular recurrent neural network architecture. Unlike conventional feedforward neural networks, LSTM has feedback connections. It can process both single input data (e.g., an image) and a sequence of data (e.g., sequences of images in a video). LSTM networks were developed to deal with the vanishing gradient problem and are well-suited for classification and prediction based on time-series data.

Figure 4.36 shows an ordinary LSTM unit, consisting of a cell, an input gate, an output gate, and a forgetting gate. The cell remembers values over arbitrary time intervals, and the three gates regulate the flow of information in and out of the cell.

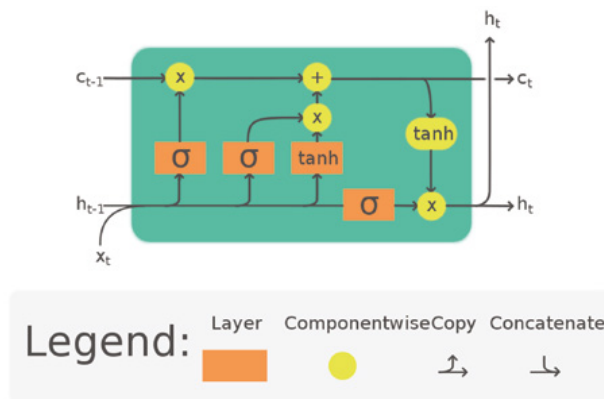


Figure 4.36: The long short memory unit structure

(Source: https://en.wikipedia.org/wiki/Long_short-term_memory)

LSTM has been commonly used in the following:

- Time-series prediction
- Time-series anomaly detection
- Speech recognition
- Language translation
- Music composition
- Text generation
- Handwriting recognition
- Human action recognition
- Robot control

There are five types of LSTM networks:

- LSTM classic
- Peephole connections
- Gated recurrent unit
- Multiplicative LSTM (2017)
- LSTMs with attention

See the following website about the five types of LSTM networks:

<https://blog.exxactcorp.com/5-types-lstm-recurrent-neural-network/>

Now, let's look at some RNN Python example code. Example 4.18 shows a simple Keras SimpleRNN example.

EXAMPLE 4.18 THE SIMPLERNN.PY PROGRAM

```
#Example 4.18 SimpleRNN.py
#https://www.tensorflow.org/api_docs/python/tf/keras/layers/SimpleRNN
import tensorflow as tf
import numpy as np

i = 4
s = 5
n = 1
inputs = np.random.random([i, s, n]).astype(np.float32)
simple_rnn = tf.keras.layers.SimpleRNN(i)
print(inputs)

output = simple_rnn(inputs) # The output has shape `[32, 4]`.
print(output)
```

This is the output of the example, showing the inputs and the outputs of the SimpleRNN:

```
[[[0.62100476]
  [0.6268874 ]
  [0.29603383]
  [0.7806354 ]
  [0.2677117 ]]]

[[[0.07438648]
  [0.4314253 ]
  [0.8583106 ]
  [0.37769878]
  [0.6659656 ]]]

[[[0.7893332 ]
  [0.52695924]
  [0.7091379 ]
  [0.7229491 ]
  [0.4395296 ]]]

[[[0.7297753 ]
  [0.9190028 ]
  [0.16167772]
  [0.24619947]
  [0.00972256]]]]
```

```
tf.Tensor(
[[ 0.74350387  0.14273308 -0.606336    0.03686696]
 [ 0.9149856  -0.29274157 -0.41164276 -0.18939926]
 [ 0.86783993 -0.00522089 -0.5399761  -0.09883117]
 [ 0.2526757   0.2733288  -0.5457421   0.54237866]], shape=(4, 4),
dtype=float32)
```

Example 4.19 is another example of Keras SimpleRNN, showing how to create an RNN model and show the model summary.

EXAMPLE 4.19 THE SIMPLERNN2.PY PROGRAM

```
#Example 4.19 SimpleRNN2.py
from keras.models import Sequential
from keras.layers import Dense, SimpleRNN, Activation
from keras import optimizers

model = Sequential()
model.add(SimpleRNN(50, input_shape = (49,1), return_sequences =
False))
model.add(Dense(46))
model.add(Activation('softmax'))

adam = optimizers.Adam(lr = 0.001)
model.compile(loss = 'categorical_crossentropy', optimizer = adam,
metrics = ['accuracy'])
print(model.summary())
```

This is the output of the previous example, showing the inputs and the outputs of the SimpleRNN:

```
Model: "sequential_1"
_____
Layer (type)                Output Shape              Param #
=====
simple_rnn_1 (SimpleRNN)      (None, 50)                2600
_____
dense_1 (Dense)              (None, 46)                2346
_____
activation_1 (Activation)    (None, 46)                0
=====
Total params: 4,946
Trainable params: 4,946
Non-trainable params: 0
_____
None
[0.2677117 ]]
```

The following is another Keras SimpleRNN example, predicting the airline passenger numbers, which is modified based on the example code at <https://www.datatechnotes.com/2018/12/rnn-example-with-keras-simplernn-in.html>.

First, the airline passenger numbers are retrieved from a website:

```
df = pd.read_csv('https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv', usecols=[1], engine='python')
```

Then put the first 80 percent of the data as training data and use the remaining 20 percent for prediction.

```
Tp = int(df.shape[0]*0.8)
```

Then build a simple RNN model, train the model with the training data (first 80 percent of the total data), predict the data, and finally plot the results.

EXAMPLE 4.20 THE SIMPLERNN3A.PY PROGRAM

```
#Example 4.20
#Modified based on:
#https://www.datatechnotes.com/2018/12/rnn-example-with-keras-simplernn-in.html
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from keras.models import Sequential
from keras.layers import Dense, SimpleRNN

# convert into dataset matrix
def convertToMatrix(data, step):
    X, Y = [], []
    for i in range(len(data)-step):
        d=i+step
        X.append(data[i:d,])
        Y.append(data[d,])
    return np.array(X), np.array(Y)

# https://www.kaggle.com/rakannimer/air-passengers
#df = pd.read_csv('AirPassengers.csv', usecols=[1], engine='python')
df = pd.read_csv('https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv', usecols=[1], engine='python')
df.head()
plt.plot(df)
plt.show()

step = 4
N = df.shape[0]
Tp = int(df.shape[0]*0.8)
```

```

values=df.values
train,test = values[0:Tp,:], values[Tp:N,:]

# add step elements into train and test
test = np.append(test,np.repeat(test[-1:],step))
train = np.append(train,np.repeat(train[-1:],step))

trainX,trainY =convertToMatrix(train,step)
testX,testY =convertToMatrix(test,step)
trainX = np.reshape(trainX, (trainX.shape[0], 1, trainX.shape[1]))
testX = np.reshape(testX, (testX.shape[0], 1, testX.shape[1]))

model = Sequential()
model.add(SimpleRNN(units=32, input_shape=(1,step),
activation="relu"))
model.add(Dense(8, activation="relu"))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='rmsprop')
model.summary()

model.fit(trainX,trainY, epochs=10, batch_size=16, verbose=2)
trainPredict = model.predict(trainX)
testPredict= model.predict(testX)
predicted=np.concatenate((trainPredict,testPredict),axis=0)

trainScore = model.evaluate(trainX, trainY, verbose=0)
print(trainScore)

index = df.index.values

plt.plot(df,label='Data')
plt.plot(predicted,label='Prediction')
plt.axvline(df.index[Tp], c="r")
plt.legend()
plt.show()

```

This is the output of the example, showing the inputs and the outputs of SimpleRNN:

Model: "sequential_1"

Layer (type)	Output Shape	Param #
=====		
simple_rnn_1 (SimpleRNN)	(None, 32)	1184
dense_1 (Dense)	(None, 8)	264
dense_2 (Dense)	(None, 1)	9
=====		

```

Total params: 1,457
Trainable params: 1,457
Non-trainable params: 0

Epoch 1/10
- 0s - loss: 41144.4444
Epoch 2/10
- 0s - loss: 17512.6174
Epoch 3/10
- 0s - loss: 8119.1773
Epoch 4/10
- 0s - loss: 4370.8642
Epoch 5/10
- 0s - loss: 2613.2462
Epoch 6/10
- 0s - loss: 2054.8084
Epoch 7/10
- 0s - loss: 1925.8166
Epoch 8/10
- 0s - loss: 1789.0628
Epoch 9/10
- 0s - loss: 1733.9799
Epoch 10/10
- 0s - loss: 1731.7953
1606.9956946331522

```

Figure 4.37 shows the output of Example 4.20, showing the original data, the predictions, and the vertical red line representing the 80 percent prediction point. The x-axis consists of data points at different time, and the y-axis consists of the number of air passengers. The predicted result after the 80 percent prediction point agrees well with the original data, even though the model has never seen the data before.

Example 4.21 shows how to create a simple Keras LSTM network. It was modified based on the TensorFlow RNN website (<https://www.tensorflow.org/guide/keras/rnn>).

EXAMPLE 4.21 THE SIMPLELSTM.PY PROGRAM

```

#Example 4.21 SimpleLSTM.py
#Based on
#https://www.tensorflow.org/guide/keras/rnn
from tensorflow import keras
from tensorflow.keras import layers
model = keras.Sequential()
model.add(layers.Embedding(input_dim=100, output_dim=32))
model.add(layers.LSTM(64))
model.add(layers.Dense(1))
model.summary()

```

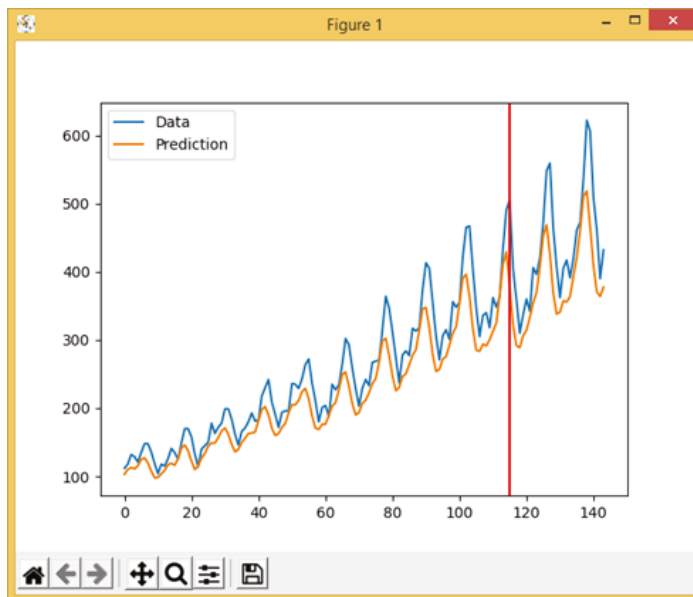


Figure 4.37: The plot output of the Example 4.17

This is the output of the example, showing the summary of the LSTM network:

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, None, 32)	3200
lstm (LSTM)	(None, 64)	24832
dense (Dense)	(None, 1)	65
Total params: 28,097		
Trainable params: 28,097		
Non-trainable params: 0		

4.4.3 Natural Language Processing and Python Natural Language Toolkit

Natural language processing (NLP) is a form of artificial intelligence that deals with human (natural) languages, in particular with the processing and analysis of large amounts of natural language data. The main challenges in NLP are speech recognition, natural language understanding, and natural language generation.

The Python Natural Language Toolkit (NLKT) is a leading open source library for natural language processing. It has easy-to-use interfaces with more than 50 corpora lexical resources (such as WordNet). In linguistics, a corpus is a language resource consisting of a large and structured set of texts. NLKT has a number of libraries for classification, tokenization, stemming, tagging, parsing, semantic reasoning, and more.

For more details about NLKT, see the following:

<https://www.nltk.org/>

Example 4.22 shows a Python example code for using the NLTK library for text analysis. The text in the code is from the Wikipedia page “Artificial Intelligence” (https://en.wikipedia.org/wiki/Artificial_intelligence).

EXAMPLE 4.22 THE NLTK.PY PROGRAM

```
# Example 4.22 The NLTK.py Program
# https://www.nltk.org/
# pip install nltk
import nltk
nltk.download('maxent_ne_chunker')
nltk.download('words')

sentence = """
Artificial intelligence (AI), is intelligence demonstrated by
machines,
    unlike the natural intelligence displayed by humans and animals.
    Leading AI textbooks define the field as the study of "intelligent
agents":
    any device that perceives its environment and takes actions that
maximize its
    chance of successfully achieving its goals.[3] Colloquially, the
term
    "artificial intelligence" is often used to describe machines (or
computers)
    that mimic "cognitive" functions that humans associate with the
human mind,
    such as "learning" and "problem solving".[4]
"""
tokens = nltk.word_tokenize(sentence)
print(tokens)
tagged = nltk.pos_tag(tokens)
print(tagged)
entities = nltk.chunk.ne_chunk(tagged)
print(entities)
```

The following are the outputs of the example, showing the tokens, tags, and entities of the text. This block shows the token entities of the text.

```
['Artificial', 'intelligence', '(', 'AI', ')', ',', 'is',
'intelligence', 'demonstrated', 'by', 'machines', ',', 'unlike',
'the', 'natural', 'intelligence', 'displayed', 'by', 'humans', 'and',
'animals', '.', 'Leading', 'AI', 'textbooks', 'define', 'the', 'field',
'as', 'the', 'study', 'of', '``', 'intelligent', 'agents', '"',
':', 'any', 'device', 'that', 'perceives', 'its', 'environment',
'and', 'takes', 'actions', 'that', 'maximize', 'its', 'chance', 'of',
'successfully', 'achieving', 'its', 'goals', '.', '[', '3', ']',
'Colloquially', ',', 'the', 'term', '``', 'artificial', 'intelligence',
'"', 'is', 'often', 'used', 'to', 'describe', 'machines', '(', 'or',
'computers', ')', 'that', 'mimic', '``', 'cognitive', '"', 'functions',
'that', 'humans', 'associate', 'with', 'the', 'human', 'mind', ',',
'such', 'as', '``', 'learning', '"', 'and', '``', 'problem', 'solving',
'"', '.', '[', '4', '']]
```

The following block shows the tags entities of the text:

```
[('Artificial', 'JJ'), ('intelligence', 'NN'), ('(', '('), ('AI',
'NNP'), (')', ')'), (',', ','), ('is', 'VBZ'), ('intelligence',
'NN'), ('demonstrated', 'VBN'), ('by', 'IN'), ('machines', 'NNS'),
(',', ','), ('unlike', 'IN'), ('the', 'DT'), ('natural', 'JJ'),
('intelligence', 'NN'), ('displayed', 'VBN'), ('by', 'IN'), ('humans',
'NNS'), ('and', 'CC'), ('animals', 'NNS'), ('.', '.'), ('Leading',
'VBG'), ('AI', 'NNP'), ('textbooks', 'NNS'), ('define', 'VBP'),
('the', 'DT'), ('field', 'NN'), ('as', 'IN'), ('the', 'DT'), ('study',
'NN'), ('of', 'IN'), ('``', '``'), ('intelligent', 'JJ'), ('agents',
'NNS'), ('"', '"'), (':', ':'), ('any', 'DT'), ('device', 'NN'),
('that', 'WDT'), ('perceives', 'VBZ'), ('its', 'PRP$'), ('environment',
'NN'), ('and', 'CC'), ('takes', 'VBZ'), ('actions', 'NNS'), ('that',
'IN'), ('maximize', 'VB'), ('its', 'PRP$'), ('chance', 'NN'), ('of',
'IN'), ('successfully', 'RB'), ('achieving', 'VBG'), ('its', 'PRP$'),
('goals', 'NNS'), ('.', '.'), ('[', '$'), ('3', 'CD'), (']', 'NNP'),
('Colloquially', 'NNP'), (',', ','), ('the', 'DT'), ('term', 'NN'),
('``', '``'), ('artificial', 'JJ'), ('intelligence', 'NN'), ('"',
'"'), ('is', 'VBZ'), ('often', 'RB'), ('used', 'VBN'), ('to', 'TO'),
('describe', 'VB'), ('machines', 'NNS'), ('(', '('), ('or', 'CC'),
('computers', 'NNS'), (')', ')'), ('that', 'IN'), ('mimic', 'JJ'),
('``', '``'), ('cognitive', 'JJ'), ('"', '"'), ('functions', 'NNS'),
('that', 'WDT'), ('humans', 'NNS'), ('associate', 'VBP'), ('with',
'IN'), ('the', 'DT'), ('human', 'JJ'), ('mind', 'NN'), (',', ','),
('such', 'JJ'), ('as', 'IN'), ('``', '``'), ('learning', 'VBG'), ('"',
'"'), ('and', 'CC'), ('``', '``'), ('problem', 'NN'), ('solving',
'NN'), ('"', '"'), ('.', '.'), ('[', 'VB'), ('4', 'CD'), (']', 'NN')]
```

The following block shows the sections of entities of the text:

```
(S
(GPE Artificial/JJ)
intelligence/NN
(/(
AI/NNP
)/)
./,
```

```
is/VBZ
intelligence/NN
demonstrated/VBN
by/IN
machines/NNS
,/,
unlike/IN
the/DT
natural/JJ
intelligence/NN
displayed/VBN
by/IN
humans/NNS
and/CC
animals/NNS
./.
Leading/VBG

... ..

functions/NNS
that/WDT
humans/NNS
associate/VBP
with/IN
the/DT
human/JJ
mind/NN
,/,
such/JJ
as/IN
``/``
learning/VBG
''/''
and/CC
``/``
problem/NN
solving/NN
''/''
./.
[/VB
4/CD
]/NN)
```

EXERCISE 4.18

Modify the previous example Python code and choose a different piece of text for text analysis. Comment on the results.

More examples about natural language processing will be available in Chapter 10.

4.5 Transformers

Transformers are new deep learning neural networks introduced by Hugging Face in 2017, mainly used in the field of natural language processing. Transformers are based on a concept called *attention*, a mechanism for weighting different parts of the input based on their importance. The creation and development of transformers has demonstrated the effectiveness of large pre-trained models for tackling NLP tasks such as machine translation and question answering. Transformers are starting to make recurrent neural networks obsolete.

Transformers consist of a number of stacked encoders that form the encoder layer, a number of stacked decoders that form the decoder layer, and a bunch of attention layers that form self-attentions and encoder-decoder attentions.

Like recurrent neural networks, transformers are designed to handle sequential data. But, unlike RNNs, transformers do not need to process sequential data in the correct order. For example, if the input data is a natural language sentence, the transformer does not need to process the beginning before the end. Because of this, transformers allow for much more parallelization and thus shorter training time.

The following are the three main types of transformers:

- **BERT:** Bidirectional Encoder Representations from Transformers
- **ALBERT:** A Lite BERT
- **GPT:** Generative Pre-trained Transformer

Transformers have been implemented in both TensorFlow and PyTorch. See the following Transformers GitHub site for details:

<https://github.com/huggingface/transformers>

4.5.1 BERT and ALBERT

BERT (Bidirectional Encoder Representations from Transformers) is a library for natural language processing. BERT was created and published in 2018 by Jacob Devlin and his colleagues at Google. Google uses BERT to better understand user search queries. The original English-language BERT model used two corpora in pre-training: BookCorpus and English Wikipedia, which together contain around 16GB of uncompressed text.

ALBERT is a “lite” version of BERT. ALBERT uses two parameter-reduction techniques to reduce memory requirements and increase training speed.

Google's ALBERT has achieved top scores on three popular benchmark tests for natural language understanding: GLUE, RACE, and SQuAD 2.0. Google has introduced three outstanding innovations with ALBERT: factorized embedding parameterization, cross-layer parameter sharing, and inter-sentence coherence loss. As a result, the large ALBERT model has about 18x fewer parameters compared to BERT-large.

Figure 4.38 shows the performance of the machine on the RACE challenge, where ALBERT has outperformed other models.

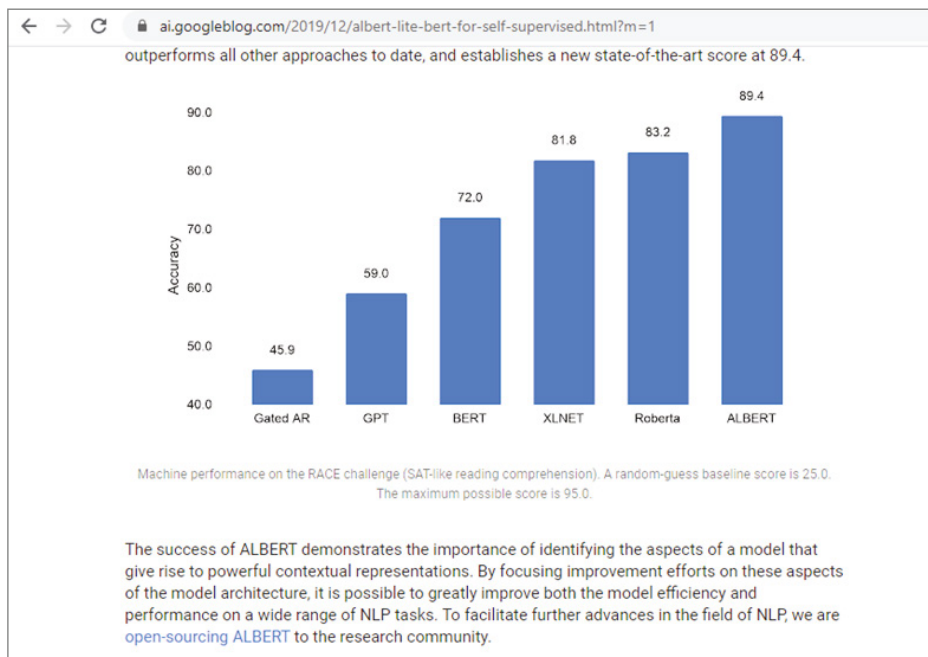


Figure 4.38: The machine performance on the RACE challenge

(Source: <https://ai.googleblog.com/2019/12/albert-lite-bert-for-self-supervised.html?m=1>)

Example 4.23 shows a Python example code of using transformers for text sentiment analysis.

EXAMPLE 4.23 THE TRANSFORMERS.PY PROGRAM

```
# Example 4.23
# https://github.com/huggingface/transformers
# pip install transformers
# https://aka.ms/vs/16/release/vc_redist.x64.exe

from transformers import pipeline
classifier = pipeline('sentiment-analysis')
classifier('This is a good movie.')
```

This is the output of the example, showing the classification result as positive:

```
[{'label': 'POSITIVE', 'score': 0.9998612999916077}]
```

EXERCISE 4.19

Modify the previous example Python code, choose some different sentences, and make sure some are positive and some are negative. Comment on the results.

Example 4.24 shows a Python example code for using transformers for question answering.

EXAMPLE 4.24 THE TRANSFORMERS2.PY PROGRAM

```
#Example 4.24
from transformers import pipeline
question_answerer = pipeline('question-answering')
question_answerer({
    'question': 'What is the name of the company?',
    'context': 'We created Biox Systems Ltd company back in the year of
2000.'
})
```

This is the output of the example, showing the answer to the question:

```
{'answer': 'Biox Systems Ltd',
 'end': 27,
 'score': 0.7553602457046509,
 'start': 11}
```

EXERCISE 4.20

Modify the previous example Python code, choose a different piece of text, and ask different questions. Comment on the results.

4.5.2 GPT-3

Generative Pre-trained Transformer (GPT) is an autoregressive natural language processing model that uses deep learning neural networks to produce human-like text. GPT was created by OpenAI, a San Francisco-based artificial intelligence research laboratory. There are three versions so far:

- GPT
- GPT-2
- GPT-3

The latest GPT-3 has 175 billion machine learning parameters and has caught the attention of the world in July 2020 with its amazing capabilities. Here is an impressive YouTube video about the GPT-3 demo:

<https://www.youtube.com/watch?v=8V20HkoiNtc&t=502s>

Before GPT-3, the largest language model was Microsoft's Turing-NLG with 17 billion parameters, released in February 2020.

There are already startups using GPT-3 to provide cool services.

Dover.io allows users to create job descriptions based on simple key words. See the following for more details about the Job Descriptions Creator - Dover.io.

<https://www.dover.io/tools/job-description-rewriter>

Fitness AI is an iPhone app that uses artificial intelligence to generate personalized workouts. Ask GPT-3 health-related or fitness-related questions for free.

<https://www.fitnessai.com/>

OthersideAI allows users to generate email messages with a list of a few bullet points.

<https://www.othersideai.com/>

Philosopher AI allows users to ask philosophical questions. You need to pay for the service.

<https://philosopherai.com/>

CopyAI helps users to automatically generate professional marketing copy based on your input. It claims to be able to automate the tedious and often frustrating aspects of copy creation.

<https://www.copy.ai/>

4.5.3 Switch Transformers

In early 2021, researchers at Google Brain developed a new, open source AI model for NLP, called Switch Transformer, which has a whopping 1.6T parameters, nearly ten times that of GPT-3. The training speed of Switch Transformer has also been improved by seven times compared to previous architectures.

Switch Transformer uses a switch feed-forward neural network (FFN) layer to replace the standard FFN layer in the transformer architecture. Instead of a single FFN, each switch layer contains multiple FFNs, called *experts*. The Switch Transformer architecture is based on the concept of Mixture of Experts (MoE). This approach simplifies the computations and reduces the communication cost.

At the time of writing this book, Google has not released the pretrained model weights for Switch Transformer; for more details about Switch Transformer, visit the following:

<https://arxiv.org/abs/2101.03961>
https://github.com/tensorflow/mesh/blob/master/mesh_tensorflow/transformer/

4.6 Graph Neural Networks

Although recurrent neural networks have been superseded by transformers in natural language processing, they are still useful in many areas that require sequential decision-making and in reinforcement learning. Recurrent neural networks typically take input data in a sequence, such as time-series data or language text. A graph neural network (GNN) is a special type of recurrent neural network that can take graphs as input data. GNN has been used in many applications such as analyzing social media data, and molecular structures. There is also the quantum graph neural networks (QGNNs) for quantum chemistry analysis. QGNNs have been applied to learning quantum dynamics, graph clustering, and graph isomorphism classification.

Convolutional neural networks work well on data with a regular grid structure, such as images, but not on graphs because they are arbitrarily large and have a complex topology.

The following website shows a friendly introduction to the GNN. It also covers the future of GNN including Quantum Graph Neural Networks (QGNNs).

<https://www.kdnuggets.com/2020/11/friendly-introduction-graph-neural-networks.html>

The following website shows the introduction to a GNN and its applications, such as modeling real-world physical systems, molecular fingerprints, protein interface prediction, modeling social interactions, and so on.

<https://neptune.ai/blog/graph-neural-network-and-some-of-gnn-applications>

The following is a GNN implementation by Google's DeepMind:

https://github.com/deepmind/graph_nets

The following is a GNN implementation by Facebook's PyTorch:

https://github.com/rustyls/pytorch_geometric

For more details about GNNs, visit the following website:

https://theaisummer.com/Graph_Neural_Networks/
<https://www.dgl.ai/>

4.6.1 SuperGLUE

SuperGlue is a CVPR 2020 research project, being conducted at Magic Leap. CVPR (Conference on Computer Vision and Pattern Recognition) is the premier annual event on computer vision, including several co-located workshops and short courses in addition to the main conference. The SuperGlue network is a GNN combined with an optimal matching layer that is trained to match two sets of sparse image features. This repo includes PyTorch code and pre-trained weights for running the SuperGlue matching network based on SuperPoint key points and descriptors. Given a pair of images, you can use this repo to extract matching features for the image pair. See the following links for details.

<https://github.com/magicleap/SuperGluePretrainedNetwork>
<https://psarlin.com/superglue/>

4.7 Bayesian Neural Networks

Traditional deep learning neural networks have a fixed value for their parameters; hence, they are also called *deterministic neural networks*. Traditional deep learning neural networks have been successful in many applications. However, they also have some drawbacks. For example, mere knowledge of the input-output mapping is inadequate when it comes to generating predictive uncertainty in their predictions. This can be important when the available data is limited or the data does not span the whole space of interest.

Bayesian neural networks (BNNs) have been developed to address these issues. Bayesian neural networks are neural networks whose weights or parameters are expressed as a distribution rather than a deterministic value and are learned using Bayesian inference. The output of Bayesian neural networks is also a distribution rather than a fixed value.

Figure 4.39 shows the differences between traditional neural networks and Bayesian neural networks.



Figure 4.39: The traditional neural networks and Bayesian neural networks

The following is the Keras website on probabilistic Bayesian neural networks, providing a detailed introduction and a Google Colab IPython Notebook with the source code. You can access the code by clicking the View in Colab link.

https://keras.io/examples/keras_recipes/bayesian_neural_networks/

Figure 4.40 shows the modified version of the Google Colab code. You can access the modified code by using the following link or by uploading a file named `Copy_of_bayesian_neural_networks_wine.ipynb` to your Google Colab.

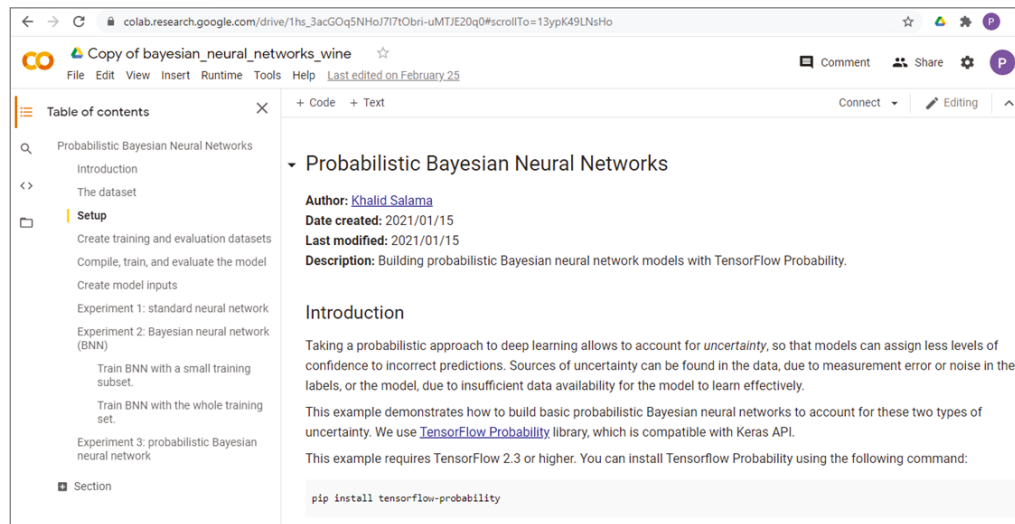


Figure 4.40: The corresponding Google Colab code for the Keras website on probabilistic Bayesian neural networks

(Source: https://colab.research.google.com/drive/1hs_3acGOq5NHoJ717tObri-uMTJE20q0#scrollTo=13ypK49LNsHo)

The modified code performs three experiments to predict wine quality.

Experiment 1: standard neural network In this experiment, the basic deterministic model is used to predict the wine quality. From the following outputs, you can see that only one fixed value is predicted for each input:

```
Predicted: 5.8 - Actual: 6.0
Predicted: 5.6 - Actual: 6.0
Predicted: 6.1 - Actual: 6.0
Predicted: 5.4 - Actual: 6.0
Predicted: 5.5 - Actual: 5.0
Predicted: 5.5 - Actual: 6.0
Predicted: 5.8 - Actual: 5.0
Predicted: 5.6 - Actual: 8.0
Predicted: 6.6 - Actual: 6.0
Predicted: 6.1 - Actual: 7.0
```

Experiment 2: Bayesian neural network (BNN) In this experiment, the Bayesian neural network model is used to predict wine quality.

The following outputs show that for each input a mean value, a minimum value, and a maximum value are predicted:

```
Predictions mean: 6.08, min: 5.47, max: 6.34, range: 0.86 - Actual: 6.0
Predictions mean: 5.46, min: 4.78, max: 5.96, range: 1.17 - Actual: 6.0
Predictions mean: 6.11, min: 5.75, max: 6.38, range: 0.63 - Actual: 6.0
Predictions mean: 5.19, min: 4.58, max: 5.81, range: 1.23 - Actual: 6.0
Predictions mean: 5.25, min: 4.76, max: 5.84, range: 1.08 - Actual: 5.0
Predictions mean: 5.15, min: 4.49, max: 5.79, range: 1.29 - Actual: 6.0
Predictions mean: 5.89, min: 5.23, max: 6.26, range: 1.03 - Actual: 5.0
Predictions mean: 5.53, min: 4.7, max: 6.15, range: 1.46 - Actual: 8.0
Predictions mean: 6.36, min: 6.12, max: 6.52, range: 0.4 - Actual: 6.0
Predictions mean: 6.02, min: 5.57, max: 6.31, range: 0.74 - Actual: 7.0
```

Experiment 3: Probabilistic Bayesian neural network This experiment uses the probabilistic Bayesian neural network model to predict wine quality. The following is the output. As you can see, the output is now a distribution, including the mean, the variance, and the confidence intervals (CI) of the prediction.

```
Prediction mean: 5.99, stddev: 0.75, 95% CI: [7.47 - 4.52] - Actual: 6.0
Prediction mean: 5.3, stddev: 0.7, 95% CI: [6.68 - 3.93] - Actual: 6.0
Prediction mean: 6.05, stddev: 0.77, 95% CI: [7.57 - 4.54] - Actual: 6.0
Prediction mean: 5.22, stddev: 0.69, 95% CI: [6.58 - 3.86] - Actual: 6.0
Prediction mean: 5.2, stddev: 0.69, 95% CI: [6.56 - 3.85] - Actual: 5.0
Prediction mean: 5.15, stddev: 0.69, 95% CI: [6.5 - 3.8] - Actual: 6.0
Prediction mean: 6.02, stddev: 0.75, 95% CI: [7.5 - 4.55] - Actual: 5.0
Prediction mean: 5.93, stddev: 0.74, 95% CI: [7.38 - 4.47] - Actual: 8.0
Prediction mean: 6.61, stddev: 0.87, 95% CI: [8.32 - 4.9] - Actual: 6.0
Prediction mean: 5.77, stddev: 0.75, 95% CI: [7.24 - 4.29] - Actual: 7.0
```

You can also plot the mean, the maximum, and the minimum on a graph. Figure 4.41 shows the code for plotting the mean, the maximum, and the minimum of the predictions (top), and the corresponding plot (bottom).

The following is a list of interesting tutorials and projects about Bayesian neural networks:

```
https://github.com/JavierAntoran/Bayesian-Neural-Networks
https://davidstutz.de/a-short-introduction-to-bayesian-neural-networks/
https://sanjaykthakur.com/2018/12/05/the-very-basics-of-bayesian-neural-networks/
http://edwardlib.org/tutorials/bayesian-neural-network
http://krasserm.github.io/2019/03/14/bayesian-neural-networks/
https://nbviewer.jupyter.org/github/krasserm/bayesian-machine-learning/blob/dev/bayesian-neural-networks/bayesian\_neural\_networks.ipynb
https://github.com/krasserm/bayesian-machine-learning
https://inferpy.readthedocs.io/en/0.0.3/notes/guidebayesian.html
```

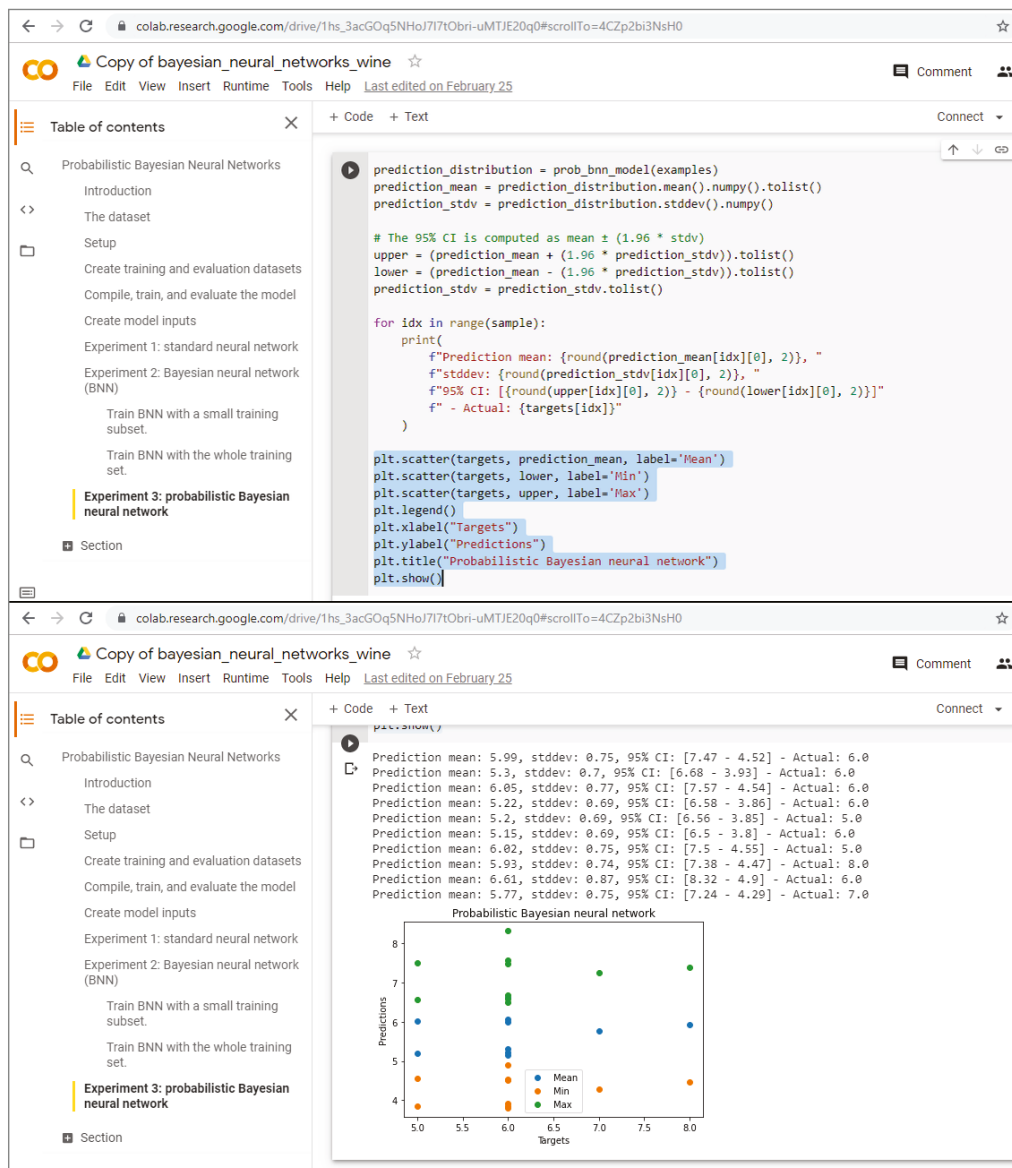


Figure 4.41: The code for plotting the mean, the maximum, and the minimum of the predictions (top), and the corresponding plot (bottom)

(Source: https://colab.research.google.com/drive/1hs_3acGOq5NHoJ7l7tObri-uMTJE20q0#scrollTo=13ypK49LNsHo)

4.8 Meta Learning

Meta learning is another approach in AI that has become popular in recent years. *Meta* is a Greek word meaning “after” or “beyond.” When used as a prefix, *meta* means “about.” So, meta learning is “learning about learning” or “learn to learn.”

The term *meta learning* was coined by Donald Maudsley in 1979, where he described a mechanism by which people are becoming “increasingly in charge of the patterns of perception, inquiry, learning, and development that they have internalized.” Later in 1985, John Biggs used the concept of meta learning to describe the state of “being conscious of one’s learning and taking control of it.” You can describe meta learning as an awareness and understanding of the process of learning itself, like thinking about thinking.

Meta learning in AI is essentially about using high-level or meta-level AI to optimize lower-level AI, so that it can learn to learn effectively and quickly, as illustrated in Figure 4.42.

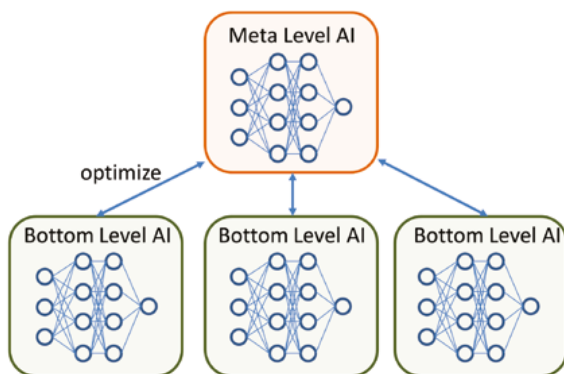


Figure 4.42: The schematic architecture of meta learning

The following are two interesting articles that give a comprehensive introduction to the meta learning, including all the mathematics behind it:

<https://jameskle.com/writes/meta-learning-is-all-you-need>
<https://lilianweng.github.io/lil-log/2018/11/30/meta-learning.html>

The following two GitHub sites give a curated list of meta learning papers, codes, books, blogs, videos, datasets, and other resources.

<https://github.com/sudharsan13296/Awesome-Meta-Learning>
<https://github.com/dragen1860/awesome-meta-learning>

The following GitHub site gives all the code examples for the book:

<https://github.com/sudharsan13296/Hands-On-Meta-Learning-With-Python>

The following two GitHub sites are meta learning projects implemented by PyTorch.

<https://github.com/learnables/learn2learn>
<https://github.com/yaoyao-liu/meta-transfer-learning>
<https://github.com/facebookresearch/LearningToLearn/tree/main/ml3>

4.9 Summary

This chapter gave a comprehensive overview of deep learning. Deep learning is the most important aspect of AI and a subset of machine learning. Deep learning is the hottest research topic in AI.

Deep learning neural networks are built on traditional neural networks, which are also called artificial neural networks. Deep learning neural networks can be generally divided into two types: convolutional neural networks and recurrent neural networks. Convolutional neural networks are the most popular deep learning neural networks, which include networks such as LeNet, AlexNet, GoogLeNet (Inception), VGG, ResNet, DenseNet, MobileNet, YOLO, and so on.

Apart from traditional convolutional neural networks, there are also new types of networks, such as U-Net, AutoEncoder, siamese neural networks, and capsule networks.

Recurrent neural networks are another popular deep learning neural network. Convolutional neural networks are specialized in processing images, while recurrent neural networks are specialized in processing sequences.

Transformers are new deep learning neural networks that are mainly used in the field of natural language processing. Transformers are gradually making recurrent neural networks obsolete.

Graph neural networks are a special type of recurrent neural network that can take graphs as input data.

Bayesian neural networks are neural networks whose weights or parameters are expressed as a distribution rather than a deterministic value. The learning of Bayesian neural networks is done by Bayesian inference.

4.10 Chapter Review Questions

- Q4.1. What is the difference between machine learning and deep learning?
- Q4.2. What is an artificial neural network?
- Q4.3. What is a convolutional neural network?
- Q4.4. Explain the terms of convolutional layer, pooling layer, activation layer, dropout layer, and fully connected layer, in the context of convolutional neural networks.
- Q4.5. Compare the features of three commonly used activation functions: Rectified Linear Unit (ReLU), hyperbolic tangent, and the sigmoid function.